# Supplementary Material: Small-State Coset Trellis Coding at IQ2_XXS Size

## Supplementary material for Appendices A–D

SA.1 to SA.6 reproduce in full the v24 text that the main paper's Appendices A–D condense.

## SA.1 What Is Borrowed and Related Work in Detail (full v24 text)

### Trellis and TCQ quantizers for LLMs: what already exists

We list what each work does, as far as its abstract, README or pull request states. **QTIP** (Tseng et al., 2024) applies TCQ with a bitshift trellis to LLM weights, with incoherence processing, and lookup-only or computed codes that decouple codebook size from the effective dimension. **EXL3** (turboderp-org, 2025) is, in its own words, a streamlined variant of QTIP with a fused Viterbi kernel on the GPU and a target-bitrate converter; it is a separate inference library, not a llama.cpp format. **The IQ*_KT types of ik_llama.cpp** (Kawrakow & ik_llama.cpp contributors, 2024) are trellis-generated quantization types inside a llama.cpp fork. The first version (pull request 113 (Kawrakow, 2024)) took the "3INST" code generator from QTIP and organised weights in llama.cpp-style blocks: IQ2_KT uses a 16-bit seed for each group of 8 weights, blocks of 32 with a 4-bit scale, and 2.125 bits per weight; the quantizer searches for seeds by clustering, with no Hadamard transform and no Viterbi search, and the author's measurement on an RTX 4080 was 194 tokens/s against 208 for IQ2_XXS (model not stated). The current version (pull request 505, superseded by the merged pull request 529 (Kawrakow, 2025a,c)) uses an integer generator whose masked bytes are summed, which, according to the pull request, allows quantized GEMM and GEMV on the GPU and much faster CPU kernels. A later pull request adds IQ1_KT at 1.75 bits per weight (Kawrakow, 2025d), which its author reports gives nearly the quality of IQ2_XXS at about 10% smaller size on Llama-3.1-8B-Instruct, with "excellent" CUDA prompt and generation speed. A contributor's measurement in the discussion of pull request 511 (Kawrakow, 2025b) gives a perplexity ratio to f32 of about 1.50 for IQ2_KT on Llama-3.1-8B-Instruct at context 512 (11.0029 against 7.3210; 10.9466 with the multiplier later merged), and that pull request reports 205.5 tokens/s generation on an RTX 4080. These are different models, contexts and hardware from ours and cannot be compared with our numbers. They do mean that a trellis type at IQ2_XXS size, with CUDA generation speed close to IQ2_XXS, exists in a llama.cpp-derived code base, so we make no such first claim; Table 7 gives the same-harness rows. **Proteus** (Yang et al., 2026) (ICML 2026) generates trellis code values by integer arithmetic and a per-layer affine transform, with no table loads at decode time, and reports results on Llama 2 7B to 70B; its public repository was a placeholder when we checked, so we could not run it. **CCQ** (Zhou et al., 2025) quantizes LLMs to 2.0–2.75 bits with a convolutional code, hybrid encoding and code clustering, using a hardware-aware bit-shift encoding and decoding and a lookup-free encoding space; it reports DeepSeek-V3 compressed to 184 GB and ERNIE-4.5-300B-A47B to 89 GB, and its 2-bit ERNIE model and inference engine are open source. Its abstract does not mention llama.cpp, and we did not run it. **Dependent quantization** (Haase et al., 2020) applies dependent scalar quantization, i.e. TCQ, with context modelling for the entropy coding of the indices to neural-network parameters, measured against the test model of MPEG-7 Part 17; dependent quantization is one of the quantization tools of the neural-network coding standard ISO/IEC 15938-17:2022 (Kirchhoffer et al., 2022), where it uses an 8-state trellis. It is not an LLM method, but it means that trellis-coded quantization of network weights is not new. **Tetra** (Malandrino, 2026) uses a 64-state trellis of the Golay code on the Leech lattice in its own engine; its Qwen3-4B file holds 2.73 bits per parameter over the whole model, scores 63.37 on full MMLU, and is reported 23.6 points above llama.cpp IQ2_XXS (2.48 bits per parameter), a larger file than ours, measured in its own engine on an L40S GPU. Its MMLU is 5-shot and read from the logits of the four answer letters (its Section 6.1), so it cannot be compared with our

0-shot continuation scores (Q8_0: 38.74, Table 16). **XOR-Trellis** (Que et al., 2026) gives a structured, low-complexity state-to-value mapping and a curvature-aware path objective in the original coordinate space, with no Hadamard-based incoherence processing. **CurveTQ** (Ding et al., 2026) removes the rotation entirely, puts the diagonal of the Hessian's LDL factor into the Viterbi branch metric, stores a start state per coding block, and reports 1 to 3 points higher mean downstream accuracy than QTIP and Proteus on three 4B to 8B Instruct models at two bits and the fastest decoder of the three. As far as their abstracts state, none of these papers targets a llama.cpp file format. Their results are on other models and in other harnesses, so we cannot rank them against our numbers, and we did not rerun them. We searched arXiv, GitHub and the web on 2026-10-14 for a combination of a small-state coset TCVQ, per-tile rate classes and a fixed-size llama.cpp file and found none (about 35 sources checked, about 16 read in depth); that is a negative search result, not a proof of absence. A GitHub search of the pull requests of ggml-org/llama.cpp for "trellis" on the same day returned no trellis quantization type.

### Bit allocation under a storage budget

Q-Palette (Lee & Song, 2025) derives an information-theoretic bit allocation and offers fractional-bit quantizers, from trellis-coded to scalar, with CUDA kernels and a mixed-scheme allocation under resource constraints. ScaleBITS (Li et al., 2026) partitions weights into hardware-aligned blocks after channel reordering and allocates bitwidths greedily under a memory budget. AlignQuant (Zhang et al., 2026) uses GPU-compatible two-dimensional tiles as the unit of precision allocation, scores tiles by prefill and decode sensitivity, and stores one representation per tile under a weight-storage budget, with INT8 compute kernels. SAGE (Yan et al., 2026) splits a fixed budget between the direction codebook and the radius precision of each matrix in a 2-bit vector quantizer. In llama.cpp itself, the standard recipes already mix types across tensors; IQ2_XXS keeps attn_v as Q4_K and four ffn_down tensors as Q2_K. Our per-tile classes choose among the 8 rate classes of one trellis code, with the same byte count for every tile of a class and a class that is uniform across a warp. We do not compare the allocation rule with those above, which differ in objective, tile shape and kernels. HAWQ (Dong et al., 2019) sets per-layer precision from Hessian spectra and BRECQ (Li et al., 2021) approximates inter- and intra-layer sensitivity for mixed precision; either could supply the $\lambda_j$ of Eq. 3, which we set to 1. SliM-LLM (Huang et al., 2024) allocates bit widths per group by salience and reports a gain from it, unlike our equal-bytes result for per-tile classes of one trellis code (Section 5.4). EvoPress (Sieberling et al., 2024) finds that per-layer error estimates assume an independence between layers that does not hold, and searches allocations end to end.

### Kernels, vector codes and evaluation

**Table-lookup kernels.** Our batch-1 GEMV, which tabulates activation–codeword dot products and then adds table entries, follows lookup-table kernels for weight-only quantization: LUT-GEMM (Park et al., 2022) on GPUs, T-MAC (Wei et al., 2024) on CPUs, and FLUTE (Guo et al., 2024), which duplicates its table to relieve shared-memory bandwidth. **Batched decoding.** MARLIN (Frantar et al., 2024) keeps 4-bit kernels close to their maximum speed-up up to batch 16–32, and QServe (Lin et al., 2024b) reports 20–90% dequantization overhead in large-batch serving; our slab path is far slower than IQ2_XXS at 2 to 32 tokens per micro-batch (0.13–0.35×, Table 13); large-batch serving was not measured. **Memory-bound generation.** SqueezeLLM (Kim et al., 2023) shows that single-batch generation is bound by memory bandwidth, and Yuan et al. (2024) analyse LLM inference with a roofline model; Table 22 shows that both 2-bit files generate far below the bandwidth bound on our GPU. **Vector codes at 2 bits.** GPTVQ (van Baalen et al., 2024) and VPTQ (Liu et al., 2024) are Hessian-guided vector quantizers with their own decoders; neither targets a llama.cpp file, and we did not run them. **Evaluation.** Dutta et al. (2024) show that compressed models of similar accuracy can change many answers and propose KL divergence and answer flips as metrics, which is why we report KL divergence beside perplexity; Kurtic et al. (2024) evaluate 8- and 4-bit formats across a model family on academic and real-world tasks.

## SA.2 Pingala's Ganas and a Note on the History of Mathematics (full v24 text)

Pingala's Chandahsutra (c. 3rd–2nd century BCE; edition with Halayudha's commentary in Pingala (1938)) opens by naming eight trisyllabic units, the ganas. Sutras 1.1–1.8 list them as *ma* (GGG), *ya* (LGG), *ra* (GLG), *sa* (LLG), *ta*

(GGL), *ja* (LGL), *bha* (GLL) and *na* (LLL), where G is a heavy (guru) and L a light (laghu) syllable; sutra 1.9 defines the symbols for G and L. The text gives these as a set of eight named triples. It gives no transitions between them. Chapter 8 of the same work gives the *prastara*, an ordered enumeration of all G/L sequences, and the rules *nashta* and *uddishta* that map between a sequence and its position. The sutras are terse; read with Halayudha's 10th-century commentary (the *Mrtasanjivani*, printed in the edition we cite), these rules are equivalent to binary place value (van Nooten, 1993; Plofker, 2009).

Write $\mathcal{G} = \{0,1\}^3$ with L$= 1$, G$= 0$, and read the first syllable as the least significant bit, so that *ma* is 000, *ya* is 001, and so on up to *na*, 111. Then the sutra order *ma, ya, ra, sa, ta, ja, bha, na* is $0, 1, \ldots, 7$, and Pingala's prastara (Chandahsutra 8.20–8.23) lists the 8 ganas in binary counting order, which is why we use them for the trellis states. A shift-register state set $\{0,1\}^\nu$ is the prastara of $\nu$ syllables: $\nu = 2, 3, 4, 5$ give 4, 8, 16 and 32 states, and the 8 ganas are the case $\nu = 3$.

We separate three objects that are often merged. (1) The ganas: a set of eight named triples. (2) The prastara with nashta and uddishta: an enumeration equivalent to binary counting. (3) The cyclic mnemonic *ya-maa-taa-raa-ja-bhaa-na-sa-la-gaa*, whose sliding 3-syllable windows visit every gana once: a de Bruijn sequence $B(2,3)$. It is not in the sutra text; Kak (2000) traces it only to Brown (1869), and Knuth (2011) calls its origin obscure. A shift-register trellis on 8 states needs (3) or an equivalent modern construction; it does not follow from (1) or (2). In this work the state set is (1), indexed by (2), and the edges and labels are Ungerboeck's (Ungerboeck, 1982). The count 8 is not load-bearing either: a 16-state trellis is better at the same kernel cost (Section 5.3), so the ancient text contributes the name and the motivation of the 8-state arm, not an optimum. The search for Pingala-related prior work did not cover Sanskrit or regional-language journals.

## SA.3 Method and Runtime Details (full v24 text)

**Viterbi recursion.** With $J_0(s) = 0$ for the reset state and $+\infty$ otherwise,

$$J_t(s') = \min_{s \to s'} \Big[ J_{t-1}(s) + \min_{i \in \mathcal{C}_{z_0(s)} \,:\, \mathrm{msb}(i) = z_1(s,s')} d_t\big(x_t, \sigma\, \mathcal{C}_{z_0(s)}[i]\big) \Big], \tag{1}$$

over the two predecessors of each of the 8 states. One step evaluates $8 \times 2$ branches, each a nearest-neighbour search over the 128 codewords with the required index MSB (8-bit stage-1 index of the 2-bit class), i.e. 2048 4-dimensional distances, or 512 per weight; the search time is $O(n\,S \cdot 2 \cdot (|\mathcal{C}|/2)/4) = O(n\,S\,|\mathcal{C}|/4)$ distance evaluations per row ($S = 8$, $|\mathcal{C}| = 256$: 512 per weight) and memory $O(S\,T)$ for the survivor paths, so the Viterbi part is linear in the parameter count and rows are independent. The Hessian part is not: the per-tensor Cholesky factorisation costs $O(n^3)$ and the error feedback $O(m\,n^2)$, which is why per-weight build time differs between tensor shapes (measured: 4623 s for the whole build of $3.63 \times 10^9$ weights on one laptop GPU, of which 2768 s in the quantiser; Appendix L extrapolates to 70B and 405B). The state is reset every 32 symbols, so the start state costs no bits.

**Decoder cost.** Decoding one symbol is: recover $u_t$ from $z_1$ and the known state (two XORs), update the state (one shift), and read one table entry; for the GEMV the table holds the 256 dot products $\langle A_k, a \rangle$ of the activation chunk $a$ with the codebook, so a base-class 4-weight symbol needs one table read and one add, plus $\langle c, a \rangle$ for the coset from subset-sum tables and integer work for the state, index and sign. Measured on sm_120 this is 278 SASS instructions per 32 class-3 symbols, 48 of them shared-memory loads (Appendix 1.4). This is $O(1)$ per weight and independent of $m$, $n$ and the model size.

**Rate classes.** Per 4 weights, stage 1 is a 4-, 6- or 8-bit index into $A$; classes at 2.5–6 bits per weight add residual stages of 2, 4 or 8 bits, so the value is $s_1 2^{\mathrm{SH}_1} + s_2 2^{\mathrm{SH}_2} + s_3$. Each tile stores one 32-bit word (payload offset and class). In Model B the 4 most salient tile-blocks per tensor are forced to class $\geq 6$, which keeps $\max|W_q| / \max|W| \geq 0.9$ on all 252 linear tensors, and a sparse fp32 delta (0.2 MB) corrects 27,437 clamped outliers. In preliminary experiments, fixed-rate codes without rate allocation lost 1.8–3.1$\times$ in importance-weighted distortion against entropy-coded scalar quantization, which motivated the per-tile classes; the equal-bytes control of Section 5.4 shows that, end to end, they do not pay.

**Byte budget of Model B.** Every one of the 252 linear tensors of Qwen3-4B is encoded. The 212 tensors that IQ2_XXS stores as IQ2_XXS get the same byte budget as the entropy-coded reference (Model A); the 40 others (36 attn_v, stored as Q4_K by IQ2_XXS, and 4 ffn_down, stored as Q2_K) get their IQ2_XXS-mix bytes. The resulting native file is 1241.6 MB in two shards, against 1246.6 MB for IQ2_XXS.

**Bit accounting.** *Linear bpw* is the stored bytes of the 252 decoder linear tensors (indices, scales, and for Model B the row permutations, column prescales and clamp deltas) times 8, over their 3,633,315,840 weights (Model B: 967.4 MB of GGUF tensor-table bytes, 2.130 bits per weight; the kernel blob alone is 2.1205). *All-tensor bpw* is the file size times 8 over all 4,022,468,096 parameters, so it includes the token embedding (Q5_K in the IQ2_XXS mix and in Model B), the norms and about 6 MB of GGUF metadata. Allocation arms are compared by *linear-blob bytes*, the logged container blobs of the 252 linears (966.3 MB for the shipped build, 972.0 MB for the flat and mirror arms), which are not GGUF tensor-table bytes.

**Token generation kernel.** Activations are quantized to int8 with one scale per 128. For each slice of 128 activations (32 chunks of 4), the kernel builds in shared memory, for every chunk, the 256 dot products between that chunk and the entries of $A$ (32 KB per thread block). Lane $j$ owns row $j$ and reads chunk $(j + k) \bmod 32$ at step $k$, which avoids bank conflicts and keeps the chunk order sequential, as the trellis needs. The coset term $\langle c, x_t \rangle$ is added from 16 two-bit subset-sum tables. Classes at 2.5–6 bits per weight need 2 or 3 reads.

**Prompt processing (PPC2).** For batches larger than one, a pre-pass kernel (K1) decodes each weight matrix once per micro-batch into an 8-bit slab in the gguf Q8_0 layout: one lane per row, shared-memory transpose, inverse Hadamard transform, $1/c$ column scale, sparse clamp delta, and the row written at its original position. The matrix multiply is then the stock llama.cpp Q8_0 MMQ kernel. Per micro-batch, K1 takes about 25 ms and the stock Q8_0 multiply on the slab about 51 ms, more than the IQ2_XXS multiplies (about 46 ms); stock Q8_0 MMQ alone ran at 0.95–1.00× IQ2_XXS, so this route cannot reach 1.00× even with a free decode. These phase times come from instrumented profiling runs (per-matmul timers, a different prompt workload, phases rescaled from 212 to 252 tensors), not from the llama-bench run, whose rate gives 88.6 ms per 512-token micro-batch (512/5777.8); the two are not additive and the phases only rank the costs. This path serves every batch larger than one, so batched decoding of several sequences also pays K1 once per step (measured for B4 in Appendix E.4, with the multi-column kernel that removes it). Peak memory during prompt processing is 56 MiB higher at depth 0 and 60 MiB at depth 4096. The CUDA patch touches 12 source files (+2954 and −102 lines).

# SA.4 Build History and Controls (full v24 text)

The body reports one file, B4. On Qwen3-4B it was preceded by four builds of the same code, each defined after the one before; this appendix gives them, the controls they provided and the pre-registered checks of B3. *Model B*, the first shipped build, gives each $32 \times 256$ tile one of 8 rate classes chosen by the weight-error proxy (Eq. 3). The flat builds give each tensor one class: *B2*, a first size-matched flat build, which tied IQ2_KT; *B3*, defined after B2, with its rule, prediction and confirmations registered before they ran (Appendix D.3); *B3+*, B3 untrimmed in a leaner layout (Appendix D.6); and *B4*, B3+ with the derived column scale (Section 3, Appendix E.4). B3, B3+ and B4 are all post hoc. In the body, "our previous build" is B3+ and "our first build" or "our per-tile build" is Model B. Table 7 gives every build with B3 as the reference; the IQ2_KT files called "knapsack, smaller", "knapsack" and "tuned" in Table 1 are the files at Model B's bytes, at B3's bytes and R1 here.

## Allocations of the earlier builds

*Model B* solves Eq. 3 per tile, forces the 4 most salient tile-blocks of each tensor to class $\geq 6$, stores a sparse fp32 delta for 27,437 clamped outliers, and gives the 212 tensors that IQ2_XXS stores as IQ2_XXS the bytes of our entropy-coded reference (Model A, Appendix J) and the 40 others (36 attn_v, 4 ffn_down) their IQ2_XXS-mix bytes: 1,241,608,288 bytes in all. The *flat control* gives every tensor one class at the nearest rate, with byte carry and salient protection off; it moves about 4.9 MB out of attn_v and in this file layout is 2.30 MB over IQ2_XXS (exact plan; 759 kB with Model B's clamp-delta size); it ships natively as B3+ (Appendix D.2). *B2* and *B3* size-match it by trimming tensors to class 2. B2 (perplexity 18.57 on the 8-bit carrier, pre-registered prediction 17.86) missed its prediction and tied IQ2_KT

(paired ΔNLL $-0.002 \pm 0.011$ (40/73 windows)). B3 then keeps the flat control for all tensors except the ffn_gate of blocks 4 and 5, which go to class 2; they were chosen, among the tensors B2 had trimmed, by the smallest increase in proxy weight error until the file was under IQ2_XXS, and this rule and the prediction (17.98, 80% interval [17.85, 18.30]) were fixed before the perplexity run (the second tensor by a pre-registered amendment; Appendix I). B3 is 1,245,915,872 bytes, 705,440 under IQ2_XXS.

## Main result of the earlier builds

B3 has far lower perplexity than IQ2_XXS and slightly lower perplexity than IQ2_KT at equal bytes, by a margin that a perplexity-tuned IQ2_KT recipe nearly closes (Table 7, Section D.3). Against IQ2_KT matched to Model B's bytes (18.62) B3 is better by $-0.030 \pm 0.011$ (55/73 windows) ($\pm 0.0165$ corrected, bootstrap $[-0.046, -0.017]$). This margin is small, B3 was chosen after B2, and B3 is 4,326,496 bytes (0.35%) larger than that IQ2_KT file, so we also built IQ2_KT matched to B3's bytes, which is slightly worse: that file reaches 18.65, and B3 is better by $-0.032 \pm 0.011$ (53/73 windows; $\pm 0.018$ after correcting for window autocorrelation, moving-block bootstrap $[-0.049, -0.018]$), inside the pre-registered $-0.024$ $[-0.036, -0.010]$. This comparison is on WikiText-2, the text on which B3 was chosen; Section D.3 gives the held-out text. B3 is also better than Model B, the per-tile build ($-0.055 \pm 0.010$ (64/73 windows)), so on this model the flat allocation is the better use of the same file size. B3's perplexity fell inside its pre-registered interval (18.07 against 17.98, 80% interval [17.85, 18.30]).

*B3+* (post hoc) is the flat control (Section D.1) as a leaner native file (Appendix D.6; prediction held, 17.7751 in [17.73, 17.86]). *B4*, also post hoc, changes one line of B3+'s front end, the input-column scale, to $c_j \propto s_j^{1/6} E_j^{-1/3}$ (importance $s_j$, column energy $E_j$; Appendix E.4). It belongs to the activation-aware channel scales of SmoothQuant and AWQ (Xiao et al., 2023; Lin et al., 2024a), in SmoothQuant's form with $\alpha = 1/3$; our derivation fixes $\alpha$ for an upper bound on LDLQ's error only, and $\alpha$ was not swept. Its rule, written before the run, was a paired win over B3+ with an interval excluding zero; it holds ($-0.066 \pm 0.009$ (70/73 windows) on WikiText-2, $-0.068 \pm 0.007$ (73/73, corrected) on C4) at 1.8 MB fewer bytes and equal speed. Against the tuned IQ2_KT (R1) its ΔNLL is $-0.094 \pm 0.010$ (71/73, corrected) on WikiText-2 and $-0.115 \pm 0.008$ (73/73, corrected) on C4 (C4-calibrated file: Table 7); it is also below the 4.6% larger default-recipe IQ2_KT (17.28). Its point prediction (17.45 [17.00, 17.85]) missed: the gain was larger. Against the strongest IQ2_KT we built at B3's bytes, B3+'s margin ($-0.021$ nats on WikiText-2, $-0.040$ on C4) is the size of one allocation choice and B3 ties it on WikiText-2.

## Pre-registered confirmation of B3

Because B3 was chosen after seeing B2, we registered five checks with decision rules before they ran; all five decision rules hold, the fourth only on its pre-registered naive intervals (Table 7), while of the point predictions registered with the first three only the two paired margins fell inside their intervals. (i) KL divergence on a 72-chunk WikiText-2 set (its first 12 chunks are the development set, the other 60 are new): B3 is claimed lower than IQ2_KT only if the difference exceeds 2 combined standard errors. It does (0.793 against 0.863). (ii) Perplexity on held-out C4 text: the WikiText-2 win counts as confirmed only if the 95% interval of the paired ΔNLL against IQ2_KT at Model B's bytes is below zero. It is: $-0.036 \pm 0.010$ (62/73 windows) ($\pm 0.011$ corrected), inside its pre-registered interval ($-0.020$ $[-0.050, +0.010]$). On C4, B3 is also better than Model B by $-0.019 \pm 0.010$ (53/73) and than IQ2_XXS by $-0.332 \pm 0.012$ (73/73). Against IQ2_KT at B3's own bytes the C4 margin is larger: 28.00 for that file, paired ΔNLL $-0.046 \pm 0.010$ (66/73 windows; $\pm 0.011$ corrected). (iii) An IQ2_KT recipe matched to B3's bytes by the same procedure as the recipe at Model B's bytes, where a 95% interval not below zero is reported as a tie: on WikiText-2, B3 beats it ($-0.032 \pm 0.011$ (53/73 windows)). (iv) Three IQ2_KT recipes tuned for perplexity at no more than B3's bytes (Setup), the best on WikiText-2 to be confirmed on C4; rule (a): B3's claim stands if both 95% intervals exclude zero, (b) a tie if either covers zero, (c) withdrawn if IQ2_KT wins. The best, R1, reaches 18.27 (predicted 19.82 [18.55, 21.90]: missed; C4 27.33). Rule (a) holds on the pre-registered naive intervals, B3 against R1 being $-0.011 \pm 0.011$ (45/73 windows; $\pm 0.017$ after the autocorrelation correction) on WikiText-2 and $-0.021 \pm 0.012$ (52/73, corrected) on C4, but the corrected WikiText-2 interval and a block bootstrap ($[-0.029, +0.003]$) cover zero, so B3's WikiText-2 margin over R1 is not robust; its C4 margin is. B3+ beats R1 under every interval (Section 4.1). (v) With (iii)'s recipe calibrated on B3's own C4 text, B3's C4 margin holds but shrinks to a third ($-0.015 \pm 0.012$ (49/73, corrected)), B3 ties on WikiText-2, and B3+ wins under every interval (Appendix E.1).

# SA.5 Earlier builds: KL divergence, zero-shot, speed and the equal-bytes proxy control (full v24 text)

## KL divergence, zero-shot accuracy and speed of the earlier builds

**KL divergence.** Perplexity and KL divergence rank the 2-bit files differently. On the 12-chunk set Model B's KL divergence (0.640; IQ2_XXS 1.220) is below IQ2_KT's at its bytes (0.785) and at 4.8% more bytes (0.668), yet IQ2_KT has 2.5% lower perplexity (Appendix E.1). On the 72-chunk set B3 ($0.793 \pm 0.010$) and B3+ ($0.769 \pm 0.010$) are below all six IQ2_KT files of this size (0.852–0.893, the perplexity-tuned R1 lowest) but above Model B ($0.695 \pm 0.009$), so perplexity favours these flat builds and KL divergence Model B; B4 ($0.585 \pm 0.008$) is lowest on both. The C4-calibrated IQ2_KT has $0.893 \pm 0.010$, $0.307 \pm 0.013$ above B4 (top-1 agreement 63.7% against 71.3%). Every file has higher KL divergence on the 72 chunks than on the 12; all four pre-registered KL prediction intervals missed (B3: 0.60 [0.54, 0.66]; this IQ2_KT: 0.845 [0.815, 0.870]), though the decision rules held.

**Zero-shot.** On the native files (Table 8) Model B beats IQ2_XXS beyond the 95% interval on all five tasks (thinly on two, which do not survive a Bonferroni correction) and ties IQ2_XS (Appendix E.2). B4 averages 50.83 (80% prediction 49.9 [48.7, 51.2]; ARC-Easy above its interval), above Model B (49.13), B3+ (48.16) and, by $4.31 \pm 1.15$ points, the C4-calibrated IQ2_KT (46.52; predicted 46.4 [45.3, 47.5]), higher on all five tasks, by about two standard errors or more on four (unpaired; ARC-Challenge $z = 0.68$; WinoGrande and ARC-Easy survive a Bonferroni correction). Against Model B, B3 averages 1.35 points lower over the five tasks (ARC-Easy $-5.6$), which misses our pre-registered expectation of a tie, and B3+ 1.0 point lower; both are above IQ2_XXS on every task (5.9 and 6.3 points on average). B3 and B3+ are 1.8 and 2.2 points above IQ2_KT at B3's bytes on average, each task within the 95% interval of the difference.

**Speed.** All files were benched in one exclusive session (Table 9; absolute rates vary by up to 46% across sessions, so we compare only within one), with all 15 pre-registered ratios inside their intervals. At batch 1, B4 decodes at 1.096 [1.034, 1.158]× IQ2_XXS and 1.102 [1.050, 1.154]× the C4-calibrated IQ2_KT (tg128, 213 tokens/s; llama-batched-bench at one sequence: only 1.020 [0.981, 1.060]); B3+, which B4 extends, is also above IQ2_XXS (1.069 [1.016, 1.123]; B4/B3+ 1.026 [0.984, 1.067]) and B3, Model B and both IQ2_KT files tie it; the B3-to-B3+ speed-up has no known cause (no CUDA change). Prompts run at 0.69–0.74×, as the stock Q8_0 multiply on the decoded slab is slower than the IQ2_XXS multiply; IQ2_KT prefills about 1.3× faster than B4. Batched decoding is a clear weakness of the shipped runtime: with 2 to 8 sequences every micro-batch decodes each matrix to the slab, and B4 runs at 0.18–0.34× IQ2_XXS (micro-batch sweep: Table 13). The post hoc multi-column kernel (each weight decoded once for all sequences) reaches 0.92, 1.06 and 1.09× IQ2_XXS at 2, 4 and 8 sequences, and 1.39–1.50× IQ2_KT.

**Other 2-bit methods.** B4 (1.336× fp16, about 2.14 bits per weight on the linears) beats a third-party LLVQ checkpoint (2.16 bits; 1.380× in llama.cpp) and QTIP 3INST as run by the LLVQ authors (2.0 bits, their harness, 1.373×), both of which beat B3+ (1.427×), but loses to the LLVQ paper's 2-gain-bit row (1.252×) and Q2_K (15.01, 34% more bytes than Model B); AutoRound (Cheng et al., 2024) w2g128 (30.26) is worse than all of our builds (Appendix E.3).

## Equal-bytes control of the weight-error proxy

At equal bytes, the allocation that the weight-error proxy prefers has the higher perplexity. Turning the per-tile classes off (one class per tensor at the nearest rate) raises the held-out weight error 1.295× as a geometric mean over the 252 linears (1.104× for the summed error). The equal-bytes per-tile arm below has proxy 1.022, so the flat arm's error is 1.27× its own (1.13× summed). To test the proxy we built a per-tile arm whose byte target for every tensor is the byte count of the same tensor in the flat build (25 kB below it in linear-blob bytes; neither carrier has the clamp delta) and fixed the prediction and the decision rule before the run (classes help if $D \leq -0.18$, tie if $|D| < 0.18$, hurt if $D \geq +0.18$, with $D$ the per-tile minus flat perplexity). The result is $D = +0.44$: 18.23 with per-tile classes against 17.79 with one class per tensor (Table 10, Figure 4), a 2.4% advantage for the flat allocation that is significant on paired per-window NLL ($-0.0245 \pm 0.0081$ nats, flat better on 51 of 73 windows). So $\lambda_j$ in Eq. 3 should be measured end to end, not read off a weight-error proxy (as in EvoPress; Sieberling et al., 2024). Layer-wise error is a known imperfect proxy for the end loss (HIGGS, GuidedQuant and YAQA; Malinovskii et al., 2025; Kim et al., 2025; Tseng et al., 2026);

ours is a measured counterexample at fixed GGUF bytes, one build per arm on one model and text, not a general law: a finer allocation that fits the proxy better can spend bytes where the output does not need them. KL divergence of the two equal-bytes arms was not measured; the shipped builds, which also differ in attn_v, salient protection and 4.3 MB, rank the other way on 72-chunk KL divergence (Model B 0.695, B3 0.793), so which end-to-end metric should drive allocation is open.

Both equal-byte arms also beat the shipped build (18.23 and 17.79 against 19.10); both move about 4.9 MB out of attn_v (from 4.4985 to 4.086 bits per weight), drop the salient-weight protection and have 0.6% more linear-blob bytes, and we did not separate these three changes (Appendix I).

# SA.6 B3+: removing layout overhead (full v24 text, with its byte and pre-registration tables)

## B3+: removing layout overhead

B3 and B3+ are rate-off builds: a scan of both blob sets finds one rate class in each of the 252 linears, with every tile word equal to its arithmetic value. In the native layout used so far (v2) two structures are then pure overhead. The 32-bit tile word (payload offset and class, Appendix C) is a function of the class and the tile index, and the row permutation (one int16 per row) only groups rows by class; rows are coded independently, so a tensor in natural row order decodes to the same rows bit for bit. B3+ takes the flat control's blobs for all 252 linears (B3 with its two class-2 trims of the ffn_gate of blocks 4 and 5 undone; the other 250 blobs are B3's files) and stores them in a new layout, VR3, without either structure.

**VR3.** Each linear is one int8 GGUF tensor: the code tables, a header word with a VR3 tag and the tensor's class, the fp16 scales and the trellis payload; there is no permutation tensor, and a key `vr.layout` $= 3$ marks the file (files without it load as before). The loader expands VR3 on the host into the exact v2 in-memory bytes (tile words recomputed from class and tile index, header word reset to the value the runtime expects, identity permutation), so no CUDA source changed and the kernels see the v2 bytes of the natural-order tensor. The row count is derived from the tensor size; sizes that do not divide, a wrong tag and class 0 are rejected.

Table 1: Byte accounting of B3+ (exact GGUF writer plan; the planner reproduces B3's measured file to the byte).

| Item | Bytes |
|---|---|
| B3, v2 layout (as evaluated) | 1,245,915,872 |
| row permutations (252 tensors, 32-byte aligned) | −2,211,840 |
| tile words (443,520 tiles × 4 bytes) | −1,774,080 |
| GGUF tensor infos and padding (252 fewer tensors, one new key) | −13,376 |
| B3, VR3 layout (layout saving 3,999,296) | 1,241,916,576 |
| blocks 4 and 5 ffn_gate back from class 2 to class 3 | +3,112,960 |
| clamp delta (219,435 entries instead of 233,221) | −110,272 |
| **B3+, VR3 layout** (996,608 under B3, 1,702,048 under IQ2_XXS) | 1,244,919,264 |
| B3+ in the v2 layout (not built; 2,297,216 over IQ2_XXS) | 1,248,918,528 |

**Exactness and determinism.** On all 252 B3+ tensors the VR3 expansion equals, byte for byte, the v2 bytes of the natural-order tensor from the shipped transcoder (144 tensors of at most 12M weights also against a one-shot transcode; 108 larger ones in whole-tile chunks), and decoding the expanded bytes with the CPU reference reproduces the decode of the original row-sorted blob on every weight. Negative controls on two tensors show the comparison detects single-byte errors (one flipped payload byte: 4 weights differ; one scale: 253–255; one class bit: 8,062–8,183; clean copy: 0). The loader's C functions match the reference expansion byte for byte on 5 dumped tensors (classes 3, 4 and 6) and reject the malformed inputs above. The clamp delta was recomputed in chunks; the recompute is bit-equal to B3's on B3's own two trimmed tensors, and the untrimmed versions have 34,081 and 8,661 entries. Both native files (B3+, and B3's weights in VR3 as a layout control) were read back tensor by tensor with 0 mismatches. On the

GPU, the old binary reproduces B3's logged first 8 WikiText-2 windows exactly (16.7182, . . . , 17.1477), and the rebuilt binary gives the same 8 values on B3's v2 file (backward compatibility) and on B3's weights in VR3 (layout).

**Pre-registration.** The design, the byte and exactness predictions, the perplexity prediction and three decision rules (file no larger than IQ2_XXS, all exactness checks pass, the GPU layout check matches B3 window for window) were written down before any B3+ scan, plan, build or run; all three rules held. Table 2 lists every prediction. The one miss is a byte prediction: B3+ in the v2 layout was predicted within $\pm 0.5$ MB of IQ2_XXS and would be 2.30 MB over (the prediction left out the 32-byte alignment of the 252 permutation tensors and underestimated the untrimmed delta), so B3+ is size-matched only because of the layout change. B3+ is post hoc in that its design followed B3, and C4 was not part of its pre-registration. Against its own 8-bit carrier (the flat arm of Table 10, 17.7897) the native file differs by $-0.0008 \pm 0.0004$ (51/73 windows), and against Model B by $-0.0715 \pm 0.0097$ (68/73). On C4, with autocorrelation-corrected intervals and moving-block bootstrap (blocks of 5), B3+ beats B3 by $-0.0254 \pm 0.0040$ $[-0.0296, -0.0227]$, IQ2_KT at Model B's bytes by $-0.0615 \pm 0.0099$ $[-0.0714, -0.0550]$ and IQ2_KT at B3's bytes by $-0.0709 \pm 0.0104$ $[-0.0815, -0.0639]$, and IQ2_XXS by $-0.357 \pm 0.012$ (73/73 windows) (26.08 against 37.28). Its 72-chunk KL divergence, $0.769 \pm 0.010$, is below those of IQ2_KT at Model B's bytes ($0.863 \pm 0.010$) and at B3's bytes ($0.876 \pm 0.010$), and of three further IQ2_KT recipes at no more than B3's bytes, pre-registered separately (0.852–0.861; Table 14). Its zero-shot accuracy is in Table 8 and its speed in Tables 9 and 13.

Table 2: B3+ pre-registration (written before any B3+ run) against the measurements. Intervals are 80% for perplexity; ΔNLL is paired over 73 WikiText-2 windows (95% interval; windows won).

| Quantity | Predicted | Measured | Hit |
|---|---|---|---|
| Layout saving on B3 | about 3.99 MB | 3,999,296 B | yes |
| B3 in VR3 | 1,241.92 ± 0.02 MB | 1,241,916,576 B | yes |
| Bytes re-spent on the two trims | +3,112,960 B | +3,112,960 B | yes |
| B3+ in VR3 | 1,244.7–1,245.1 MB | 1,244,919,264 B | yes |
| B3+ under B3 | 0.8–1.2 MB | 996,608 B | yes |
| B3+ under IQ2_XXS | 1.5–1.9 MB | 1,702,048 B | yes |
| Clamp-delta change | −0.38 to −0.02 MB | −0.11 MB | yes |
| B3+ in v2 layout vs. IQ2_XXS | within ±0.5 MB | +2.30 MB | **no** |
| VR3 expansion, decode | bit-exact, 252/252 | 252/252 | yes |
| WikiText-2 perplexity (native) | 17.78 [17.73, 17.86] | 17.7751 | yes |
| ΔNLL vs. B3 | $< 0$, mean $-0.016$ | $-0.0162 \pm 0.0028$ (69/73) | yes |
| ΔNLL vs. IQ2_KT, Model B's bytes | $< 0$ | $-0.0462 \pm 0.0099$ (64/73) | yes |

# Supplementary material: full text of shortened appendix passages

The main paper's appendix gives shortened versions of the passages below; here each passage is reproduced in full, unchanged. Numbers, intervals, predictions and verdicts are identical to those in the main paper.

## Additional Results on Qwen3-4B: composition of the speed and energy ratios (Table 11)

Speed and energy ratios are medians of per-round ratios, so they need not compose: for pp512 at depth 0 the per-round energy ratios are 4.10, 1.70 and 2.05 (4 to 9 power samples per run), and in the round that gives the median 2.05 the throughput ratio is 0.620 and the board-power ratio 1.27; the throughput median 0.702 comes from another round (supplementary material).

## Stock-quant curve: full discussion

At the IQ2_XXS file size, Model B reaches the perplexity of IQ2_XS, which is 112.5 MB (9.1%) larger and 0.4% lower in perplexity (19.02 against 19.09), while Model B is lower in KL divergence (0.640 against 0.773) and higher in top-1 agreement (68.7% against 63.2%). Q2_K (1669.5 MB, 34% larger than Model B) is better on all three measures. IQ1_M is smaller (1127.0 MB) and much worse. IQ2_S, IQ2_M and IQ3_XXS are not in the table: in our CUDA build

these three stock files produce garbage (WikiText-2 perplexity $2.4 \times 10^6$ to $3.4 \times 10^6$, KL divergence above 13 nats, 0.0% top-1 agreement), while IQ1_M, IQ2_XXS, IQ2_XS and Q2_K from the same build are sane. A CPU-backend run of IQ2_M gives perplexity 18.63 on two context-512 chunks (the only cross-check), so we attribute the failure to a runtime defect on this GPU build, not to the files. It is unresolved and we report no numbers for these three formats.

## Stock-quant speed (shared-GPU session; moved from the main appendix)

Table 3: Speed of the stock quants against IQ2_XXS on Qwen3-4B at depth 0. Each cell is the median over 3 rounds of the same-round ratio to IQ2_XXS, with the range over rounds in brackets. The GPU was shared during this run, so the ratios are indicative; the Model B row of Table 11 is the one to quote.

| Format | tg128 vs. IQ2_XXS | pp512 vs. IQ2_XXS |
|---|---|---|
| IQ1_M | 1.08 [0.98, 1.18] | 0.83 [0.75, 0.86] |
| IQ2_XXS | 1 | 1 |
| Model B (native file, same run) | 1.05 [1.04, 1.10] | 0.70 [0.64, 0.71] |
| IQ2_XS | 1.03 [0.99, 1.14] | 1.12 [1.12, 1.29] |
| Q2_K | 0.89 [0.86, 1.08] | 0.76 [0.70, 0.82] |
| Q8_0 | 0.65 [0.53, 0.69] | 1.03 [1.02, 1.07] |

## IQ2_KT at default and at Model B's bytes: full paragraph

**IQ2_KT at default and at Model B's bytes.** The fork's default IQ2_KT recipe keeps 40 tensors in IQ3_K and 37 in IQ4_K and produces 1300.8 MB, 4.8% more than Model B; there it reaches perplexity 17.28 (1.387×), KL divergence 0.668 and top-1 agreement 67.5%. The recipe at Model B's bytes (205 tensors in IQ2_KT, 25 in IQ3_K, 23 in IQ4_K) was chosen by a knapsack over the tensors the fork upgrades, with a calculated (not measured) sensitivity proxy. At equal bytes IQ2_KT has 2.5% lower perplexity than Model B (18.62 against 19.09) and prefills 1.57× faster in this session (paired pp512 ratio 1.571 [1.537, 1.601]; about 1.3× in both exclusive-GPU sessions, 0.948 against 0.748 below and 0.92 (IQ2_KT at B3's bytes) against 0.70 in Table 9); Model B has 18% lower KL divergence (0.640 against 0.785) and higher top-1 agreement (68.7% against 64.1%); generation speed ties (paired tg128 ratio 0.976 [0.948, 1.001]). A later session with the GPU held exclusively by the benchmark gives the same ordering: against IQ2_XXS, Model B 0.967 [0.872, 0.982] in tg128 and 0.748 [0.677, 0.811] in pp512, IQ2_KT 1.001 [0.862, 1.033] and 0.948 [0.915, 0.997] (cross-binary for IQ2_KT). These are single runs on one model and one text; the recipe is one point, the ik binary and our patched llama.cpp are different builds (about 1% speed offset on IQ2_XXS; perplexity of the same IQ2_XXS file 28.6037 in ik_llama.cpp against 28.6083 in mainline), and the KL divergence uses a Q8_0 reference regenerated with mainline llama.cpp. Size curve (calculated): with ln perplexity linear in file bytes through the default recipe (1,300,786,816 bytes, 17.2750) and the recipe at Model B's bytes (1,241,589,376 bytes, 18.6165), IQ2_KT would reach B3's 18.0660 at about 1,265.3 MB, 19.4 MB more than B3's 1,245.9 MB, or 1.56% of B3's bytes (margin bytes over B3's file bytes); a log-log fit gives 19.1 MB (1.53%). The recipe at B3's bytes (18.6481) lies above this curve and is not used in the fit; the perplexity-tuned R1 (18.2721 at 240,736 bytes under B3) lies well below it, so the curve overstates B3's margin in bytes.

## Perplexity-tuned IQ2_KT recipes at B3's bytes: full paragraph

**Perplexity-tuned IQ2_KT recipes at B3's bytes.** After B3, three IQ2_KT recipes at no more than B3's bytes were written down with predictions and a decision rule before any of them was quantized (Table 14); each was quantized from the fp16 file with the same importance matrix and its byte count matched the plan exactly. R1 applies the fork's own layer heuristic (`use_more_bits`) to choose which tensors drop to IQ2_KT, R2 solves the knapsack of the recipe at B3's bytes with measured instead of calculated per-tensor error, and R3 copies B3's per-tensor shape. The selection rule picks the best on WikiText-2 (R1) and confirms it on C4. A fifth file, registered with predictions before it was quantized, tests whether B3's held-out C4 margin is a calibration-domain effect: the recipe at B3's bytes with an importance matrix computed on B3's own C4 calibration text (the first 512 sequences of 2048 tokens, Q8_0 model). It is better than we predicted on both texts (WikiText-2 18.15 against 18.75 [18.45, 19.05]; C4 27.15 against 27.70 [27.30,

28.05]); against it B3 is $-0.005 \pm 0.017$ (40/73 windows, corrected) on WikiText-2 (a tie; moving-block bootstrap $[-0.022, +0.009]$) and $-0.015 \pm 0.012$ (49/73, corrected) on C4 (bootstrap $[-0.026, -0.004]$), a third of its C4 margin over the same recipe with the default importance matrix, so most of that margin was a calibration-domain effect. B3+ beats it by $-0.021 \pm 0.015$ (52/73, corrected) (bootstrap $[-0.037, -0.009]$) and $-0.040 \pm 0.011$ (63/73, corrected) ($[-0.051, -0.031]$). Our predictions, from a per-tensor error proxy calibrated on the recipe at Model B's bytes, ranked R1 last; it came first, 1.55 below its predicted median, and outside its interval on C4 as well (predicted 29.76 [27.85, 32.88]).

## Zero-shot harness: full paragraph

**Harness.** Every zero-shot score in this paper comes from the task modes of llama.cpp's `llama-perplexity` (`-hellaswag`, `-winogrande`, `-multiple-choice`): 0-shot, with each answer scored by the log-likelihood of its continuation. HellaSwag uses 1000 items of the validation file, as selected by `llama-perplexity`, WinoGrande the debiased evaluation file (1267 items), ARC-Easy and ARC-Challenge their validation splits (570 and 299), the MMLU subset llama.cpp's MMLU validation file (1548) and full MMLU the test set (14,042). These numbers are therefore far below MMLU scores from other protocols: Tetra reports 63.37 for its Qwen3-4B file with 5 shots and answer-letter logits (Appendix A.1), and the LLVQ authors report 70.2 for unquantized Qwen3-4B in their harness, against 38.74 for Q8_0 here. Differences between files in one harness are comparable; levels across harnesses are not. Differences against IQ2_KT are unpaired.

## Comparison with other 2-bit methods: full discussion

The third-party LLVQ checkpoint (decoded bit-exactly with its author's decoder into our HF fp32 harness) reaches $1.371\times$ that harness's fp16 at 2.1595 bits per weight on the linears, against $1.533\times$ at 2.130 for Model B in llama.cpp: 0.112 nats per token better at a rate 0.029 bits per weight (1.4%) higher. On its 8-bit carrier in llama.cpp it gives 17.1905 ($1.380\times$), 0.105 nats better than Model B. We have not measured how much of that gap the extra rate would close. The checkpoint sits at the LLVQ paper's no-fine-tuning 0-gain-bit row (17.05, 1.374) and QTIP 3INST row (17.04, 1.373), not its 2-gain-bit row (15.54, 1.252). Its card states one gain bit, an input-side rotation, C4 calibration of 64 windows of 2048 tokens and no fine-tuning; the paper calibrates on 6,100 DCLM-edu sequences. We did not test which difference explains the gap to 15.54. The GPTQ w2g128 checkpoint ($25.4\times$) is consistent with GPTQ failing at 2 bits on Qwen3-4B: the Qwen3 quantization study reports WikiText-2 perplexity 195 for GPTQ w2g128 against 13.7 for fp16 at context 2048 (Zheng et al., 2025, Table 4; a different harness, so only the order of magnitude compares). QTIP, QuIP# and AQLM were not rerun: we did not run their official code on Qwen3-4B, and our QTIP port was not run to completion; QTIP 3INST on Qwen3-4B was run by the LLVQ authors (17.04 without fine-tuning). Their fine-tuned row (9.61) is below their fp16 12.41 at the same context of 4096: their optional step trains per-column input scales on about 52M tokens, which they call a lightweight alignment step, and every fine-tuned Qwen3-4B row of their table is below fp16, so those rows reflect adaptation to text, not only quantization error, and we compare only with rows without fine-tuning.

## B4: full derivation and discussion of the column scale

B3+ prescales input column $j$ by $c_j = \sqrt{\max(s_j/\bar{s}, 10^{-2})}$, with $s_j$ the importance-matrix entry. We derive the scale instead. Model the Hessian as diagonal ($H_{jj} \propto s_j$) and let $E_j = \sum_r W_{rj}^2$ be the column energy. After the 256-point rotation the scaled weights of a block $b$ are close to i.i.d., so the coding error per scaled weight is proportional to the block's mean energy $\frac{1}{B}\sum_{j\in b} c_j^2 E_j$, and column $j$'s share of the output error is that error times $s_j/c_j^2$. QuIP's incoherence upper bound on LDLQ's error (Chee et al., 2023) replaces the mean of $s_j/c_j^2$ by $(\frac{1}{B}\sum \sqrt{s_j}/c_j)^2$, so the proxy loss of block $b$, an upper bound rather than the error itself, is

$$L_b(c) = \Big(\tfrac{1}{B}\sum_{j\in b} c_j^2 E_j\Big)\Big(\tfrac{1}{B}\sum_{j\in b}\frac{\sqrt{s_j}}{c_j}\Big)^2 \;\geq\; \Big(\tfrac{1}{B}\sum_{j\in b}(s_j E_j)^{1/3}\Big)^3, \tag{2}$$

by Hölder's inequality with exponents $(3, \frac{3}{2})$, with equality iff $c_j^3 \propto \sqrt{s_j}/E_j$, that is $c_j \propto s_j^{1/6} E_j^{-1/3}$. The minimum is the power mean of order $\frac{1}{3}$ of $u_j = s_j E_j$, the same cube-root form as the optimal point density of high-rate scalar quantization (Panter & Dite, 1951); B3+'s scale ignores $E_j$. The scale is of the activation-aware per-channel family of

SmoothQuant, whose migration scale is an activation magnitude to the power $\alpha$ over a weight magnitude to the power $1-\alpha$ (Xiao et al., 2023), and AWQ, which searches a power of the activation magnitude (Lin et al., 2024a): $s_j^{1/6} E_j^{-1/3}$ is that form with $\alpha = 1/3$ and RMS in place of maximum magnitudes. Our derivation fixes $\alpha = 1/3$ only for the bound $L_b$. The $\alpha = 1/2$ form is QuIP's minimiser of its proxy $\mathrm{tr}(H)\|W\|_F^2$ (Chee et al., 2023, Supp. B.1), which Narita & Sato (2026) obtain for W4A4 activation quantization; with error feedback the scale acts on LDLQ's exact error through the Cholesky pivots of the scaled Hessian (Ben David et al., 2026; Lifar et al., 2026; Ordentlich & Polyanskiy, 2026), so the bound's exponent is not shown optimal for the error. We did not sweep $\alpha$, so a different exponent may give lower perplexity. B4 uses this scale, normalised to geometric mean 1 and clipped to $[1/64, 64]$. It is stored where B3+'s scale is (one fp16 value per column), so the file layout is unchanged, and it is the only change: same tables, rate classes, damping and LDLQ, with 256 calibration sequences instead of 512 (disk limit), which we expect to hurt rather than help B4 (not tested).

## B4: pre-registration and result, full paragraph

**Pre-registration and result.** Before the build we wrote down the prediction (17.45 [17.00, 17.85]) and the rule: B4 replaces B3+ only if its paired $\Delta$NLL against B3+ is negative with a 95% interval excluding zero (supplementary material). On two layers (14 tensors) the held-out output error fell to 0.46× that of B3+'s front end; over all 252 linears to 0.73× (geometric mean; one tensor worse). The full file reaches 16.64 on WikiText-2 (below the interval: the prediction missed), 24.36 on held-out C4 and a 72-chunk KL divergence of $0.585 \pm 0.008$, with paired $\Delta$NLL $-0.066 \pm 0.009$ (70/73 windows) (WikiText-2) and $-0.068 \pm 0.007$ (73/73, corrected) (C4) against B3+, and $-0.542 \pm 0.020$ (73/73 windows) and $-0.425 \pm 0.013$ (73/73, corrected) against IQ2_XXS. Speed equals B3+'s (tg128 1.007×, pp512 1.009× (median of 3 interleaved rounds)). The cube-root scale is optimal for the bound $L_b$; it is not shown optimal for LDLQ's exact error or for perplexity. Across 16 Qwen3-4B builds the summed output error ranks perplexity only loosely (Spearman 0.63, leave-one-out error 0.054 nats), although it ranks the KL divergence of the four builds we measured correctly and predicts the B3+→B4 step ($-0.063$ predicted, $-0.067$ measured); it does not detect the Llama failure of Appendix F (supplementary material).

## B4: same-path evaluation, full paragraph

**Same-path evaluation.** Perplexity and KL divergence elsewhere in this paper use the slab path and generation speed the batch-1 GEMV (Section 3). For B4 we also evaluated on the GEMV path itself (micro-batch 1), so that its quality and speed come from one kernel: perplexity 16.68 against 16.64 on the slab path (+0.26%; paired $\Delta$NLL $+0.0026 \pm 0.0007$, 73 windows), KL divergence 0.586 against 0.585 (72 chunks), at 182.8 tokens/s, 1.11× IQ2_XXS and IQ2_KT at B3's bytes (per-round paired ratios 1.109 [1.099, 1.119] and 1.106 [1.094, 1.117], 10 exclusive rounds in a B4-only session; 1.096 against IQ2_XXS in the one-session bench of Table 4); on that path it stays below the tuned IQ2_KT by $-0.091 \pm 0.010$ (72/73 windows). B4 does not show B3's batch-1 gap (Appendix 1.4): +0.26% over 73 windows, with 11 clamp-delta entries in the seven early ffn_gate tensors against B3's 148,300. It shares the batched weakness: at 2, 4 and 8 parallel sequences its aggregate decode rate is 0.18, 0.26 and 0.35× IQ2_XXS (llama-batched-bench, in the session of the next paragraph; 0.19, 0.27 and 0.35× in an earlier session), since only the batch-1 GEMV avoids the slab.

## B4: multi-column decoding (post hoc), full paragraph

**Multi-column decoding (post hoc).** With 2 to 8 sequences the shipped runtime takes the slab path, whose pre-pass decodes the whole model every step (34.3–34.8 ms per step at 2, 4 or 8 sequences, against 5.53 ms for the batch-1 GEMV). We added a GEMV for 2, 4 and 8 activation columns that decodes each weight group once, with the same trellis and coset decode as the batch-1 kernel, and accumulates into every column; larger batches keep the slab path. Correctness: on all 252 tensors and 2 to 8 columns, every column equals the batch-1 kernel bit for bit where both sum in a fixed order (256-column blocks), and within $6.3 \times 10^{-7}$ relative otherwise (float atomics). Perplexity on 8 windows moves by at most 0.0027 nats per window, inside twice the shipped kernel's own run-to-run spread (0.0018); our pre-registered gate of 0.0005 was below that spread, so this acceptance is post hoc. In one session (llama-batched-bench, median of 3) aggregate decode at 1, 2, 4 and 8 sequences is 158.5, 229.2, 377.0 and 561.7 tokens/s for B4 against 139.4, 240.6, 338.2 and 496.1 for IQ2_XXS (1.14, 0.95, 1.11, 1.13×) and 156.3, 43.8, 87.3 and 173.6 for the shipped B4 runtime; in the later one-session bench with all files (Figure 3b) the kernel gives 0.92, 1.06 and 1.09× IQ2_XXS at 2, 4

and 8 sequences. All rates in the first multi-column session were about 20% below the earlier session's, and IQ2_XXS's batch-1 rate ranges from 139.4 to 203.1 tokens/s across our sessions (46%), so we report only same-session ratios; the predicted absolute rates (supplementary material) were met at 8 sequences only.

## B4: trellis ablation, full paragraph

**Trellis ablation.** To measure what the Pingala-8 trellis contributes inside B4, we rebuilt B4 with only the trellis memory removed: the encoder runs the one-state greedy LDLQ path, the decoder's coset vector is zero, and the codebook is our memoryless 256-point four-dimensional table (8 bits per 4 weights, as in B4). Everything else is B4's: column scale, rate classes, Hessians, calibration, clamp rule, non-linear tensors, layout and binary. The blob bytes equal B4's exactly (972,043,712 B, all 252 tensors) and the native file is 2,880 B smaller (fewer clamp entries). One limit: the memoryless table's own residual table for later stages cannot be stored in the coset container, so all stages read the first table, as they do in B4. Before the build we predicted $+0.045$ [$+0.025$, $+0.075$] nats (90% interval) against B4 from the source-level gap and the measured error-to-NLL slope, and wrote the rule: the trellis counts as load-bearing if the 95% interval excludes zero, and as the main source of B4's gain only if the cost exceeds 15% of B4's gain over IQ2_XXS (supplementary material). The weighted output error rose 11.7% (predicted 8–14%). The memoryless file reaches 17.96 on WikiText-2 (73 windows) against 16.64 for B4 re-measured with the same binary (identical to the original run), a paired $\Delta$NLL of $+0.076$ [$+0.062$, $+0.090$] (lag-1 corrected), with B4 better on 70 of 73 windows (Figure 4c). The prediction missed (the interval's upper end was 0.075). The trellis is therefore load-bearing but not the main source of B4's gain over IQ2_XXS (14%, $-0.466$ for the memoryless file); against the tuned IQ2_KT, the memoryless file keeps only $-0.018 \pm 0.017$ (45 of 73 windows) of B4's $-0.094$ (both on the slab path, $-0.094 \pm 0.010$ (71/73, corrected)), so the trellis supplies about 80% of that margin.

## Llama-3.1-8B: the failed flat build, full paragraph

We applied the flat rule blind to Llama-3.1-8B (base) with frozen scripts and a pre-registration written before the build finished (predicted perplexity of this flat build 9.6 [8.6, 10.9] on 70 WikiText-2 windows; IQ2_XXS measured 12.35). The native file (2,389,851,808 bytes, 9.36 MB under IQ2_XXS) and its Q8_0 carrier both fail: the first window scores 11.19 against 12.14 for IQ2_XXS, but from the second window on the perplexity diverges (cumulative 6,490 after two windows). A per-tensor comparison with the Q8_0 weights finds 5 of the 224 linear tensors badly encoded, all attention projections of layers 0 and 1, whose relative Frobenius error exceeds the weights' own norm (layer 0 query 6.82, key 6.24, output 1.19; layer 1 query 1.80, key 1.93; IQ2_XXS on the same tensors 0.61, 0.62, 0.37, 0.40, 0.41; mean over all tensors 0.44). The build's Hessian-weighted statistics rated these tensors better than IQ2_XXS (layer 0 query: weighted error 0.44 of IQ2_XXS's), because with damping 0.1 of the mean Hessian diagonal the encoder placed the error in input directions that the C4 calibration text barely excites; WikiText-2 excites them. By the pre-registered rules the flat rule does not transfer to Llama-3.1-8B as built (D3). These tensors are not the whole cause (re-encoding them does not remove the failure, below); we report the build as a failure of a calibration that never shows the encoder a BOS-less sequence, and we now check every tensor's unweighted error against the reference before any evaluation. Post hoc, we traced the failure to the protocol's BOS-less windows: llama.cpp's strided mode writes the BOS only after decoding, so every window after the first starts with an ordinary token as the attention sink, while every calibration sequence of our build starts with a BOS. Without a BOS the first-token massive activation does not form in our file (median peak activation 74 against 393 for Q8_0; IQ2_XXS is close to Q8_0). With a BOS at every chunk (standard llama.cpp perplexity) on 20 windows of WikiText-2 *validation* text, the same pre-registered file scores 10.16 against 11.29 for IQ2_XXS, but IQ2_KT at the same budget scores 9.27 there and beats it. Re-encoding the five worst tensors with Frobenius weighting does not remove the BOS-less failure (70-window test perplexity 80,206). IQ2_KT at the same budget (2,389,350,432 bytes, from the Q8_0) reaches 10.04 on the pre-registered protocol (IQ2_XXS 12.35), so it is the bar a robust build has to beat.

## Llama-3.1-8B: B4 (pre-registered), full paragraph

**B4 on Llama-3.1-8B (pre-registered).** After the flat build failed we applied B4, the Qwen recipe with the cube-root column scale, to Llama-3.1-8B with the same build engine and evaluation protocol and one calibration change, also used by the intermediate build: every second calibration sequence starts mid-text without a BOS, so that the encoder sees the plain-token attention sink of the evaluation windows (Qwen3-4B's calibration has a BOS on every sequence).

We wrote down before the build the prediction (9.85 [9.50, 10.25]) and the rule: B4 counts as a win only if its paired ΔNLL against IQ2_KT at the Llama budget (2,389,350,432 bytes, built from the Q8_0) is negative with a 95% interval excluding zero. The native file has 2,388,210,816 bytes (1,139,616 fewer than IQ2_KT) and puts 32.3 MB more into the token embedding and 33.5 MB less into the linear layers than IQ2_KT. On 70 WikiText-2 windows it reaches 9.14 (IQ2_KT 10.04, IQ2_XXS 12.35, an intermediate build with the column scale in layers 0–1 only 10.38): paired ΔNLL $-0.094 \pm 0.005$ against IQ2_KT, $-0.301 \pm 0.010$ against IQ2_XXS and $-0.127 \pm 0.006$ against that intermediate build, each better on 70 of 70 windows (intervals corrected for lag-1 autocorrelation). The prediction missed: the result lies below its interval. On the BOS-less sink-start check window, batch-1 and batched decoding agree within 0.0013 nats, and B4 is 0.068 nats below IQ2_KT there. Two details of the comparison (the three caveats are in Section 4.3): IQ2_KT was quantized from the Q8_0, which on Qwen3-4B we estimate costs it about 0.026 nats against a bf16 source, well below the margin; and the build ran in six runs, five of them resumed after memory stops by replaying the finished layers. A post hoc hybrid with B4 in layers 0–17 only reached 9.54, so the full file is 0.043 nats better than the half.

## CPU backend: full section text

The native file also runs on the llama.cpp CPU backend with an AVX2 kernel. Decoding is bit-equal to the reference transliteration on all weights of 28 tensors. On WikiText-2 (context 512, 20 chunks) the CPU batch path gives perplexity 25.4586 against 25.4532 for the exact CUDA path (+0.021%), and the token-generation path (batch 1) gives 25.4672 (+0.055%); IQ2_XXS gives 31.86 on the CPU. Speed is a loss. On an Intel Ultra 9 275HX with 8 threads and three paired rounds against IQ2_XXS (19.9–21.2 tokens/s), token generation runs at 0.56× with the activation precision that passes the perplexity gate (14-bit activations as two int8 planes) and at 0.62× with 8-bit activations, which fails the 0.1% gate (+0.48% perplexity in batch mode). Prompt processing at 16 threads is 0.90× in 8-bit mode and 0.48× in the exact mode. The limit is the format: 0.30 gathered table elements per weight against one 8-byte grid load per 8 weights for IQ2_XXS, which gives a ceiling of about 0.7× at 8 threads; reaching 0.9× would need a codebook with 8 weights per lookup, a format change. With the cheapest activation mask that passes the batch gate the token-generation path gives 25.4503, level with the exact CUDA path; a final paired speed run with that mask was not stable (per-round ratios 0.54, 0.74 and 0.36), so we report no mask-tuned ratio. For phones the NEON path compiles for aarch64 and passes an emulator check but has never run on hardware; an instruction-count estimate (calculated, not measured) gives 0.77–0.93× the IQ2_XXS generation speed.

## Ablations in detail: full text of the state-count, rate-class and attn_v paragraphs

**State count.** The 16-state trellis is 1.7% better in perplexity than the 8-state one (18.78 against 19.10), although its importance-weighted tensor distortion is only 0.45% lower (geometric mean over the 212 IQ2_XXS-slot tensors). The weight-error proxy is monotone in the number of states: 4 states are 1.3% worse than 8, and 16 and 32 are 0.5–0.6% better. The full-model perplexity agrees in direction and is larger: 4 states give 20.14, 5.4% above the 8-state build, at the same linear-blob bytes (the 4-state arm came from a later build script and no 8-state arm exists in that script, so this 5.4% carries that caveat). No 32-state perplexity exists (that build was killed and not rerun). In a same-run standalone GEMV microbenchmark of 15 rounds (the seven linear shapes of one Qwen3-4B layer at about 2.06 bits per weight, GEMV time only) the 4-, 8- and 16-state kernels took 0.8445, 0.8496 and 0.8591 of the time of llama.cpp's IQ2_XXS GEMV on the same shapes, a spread of 1.7% inside round-to-round noise (individual rounds from 0.80 to 0.94). This kernel-time ratio is not an end-to-end speed: tg128 also runs the higher rate classes (2 or 3 reads per symbol), attention, norms and the output head, and is about 1.0 (Table 4). In the weight-error proxy 8 states capture about 68% of the improvement from 4 to 32 states. We ship 8 states because that is the build that exists as a native file with measured size, speed and bit-equal decoding; the 16-state result is a carrier-based control.

**Per-tile rate classes and the rate-off arm.** The 1.73× in Table 18 is the geometric mean over 21 tensors of layers 0–2 only (6.4× on ffn_gate there) and does not represent the model: over all tensors the rate-off arm has the lower error on 50 of 252 tensors, the 20 tensors with the largest error tie (0.997×), and over all 252 linears the factor is 1.295× (1.261× on layers 3 to 35). This is a weight-error proxy only, measured with Hessians from a different corpus than the perplexity text. The rate-off build alone does not test the classes, because it is not at the same bytes: its linear-blob bytes are 5.8 MB (+0.597%) above those of the shipped build and its whole file is 759 kB larger than IQ2_XXS, because rounding each tensor to the nearest class raised the index budget (typically from 1.971 to 2.000 bits per weight) on the 212 IQ2_XXS-slot tensors (+1.28% bytes) while attn_v lost 9.2% of its bytes; it also has the byte carry and the

salient-weight protection switched off. The mirror arm uses the standard per-tile allocator (byte carry on, no salient floor) with the byte target of every tensor set to the byte count of the same tensor in the rate-off build; its linear-blob bytes are 25 kB ($-0.0025\%$) below those of the flat build. Its pre-registered prediction was 18.3 with an 80% interval of 17.4 to 19.1, which contains the result (18.23). The size-matched flat builds that follow are B2 (18.57, carrier; prediction 17.86 missed) and B3 (18.07, native file; Section D.1); B3+ (17.78, native file; Appendix D.6) is the rate-off build itself in a leaner layout. B3's second trimmed tensor was added by a pre-registered amendment, before any perplexity run, as the one-tensor plan was 829,824 bytes over IQ2_XXS (that plan stores 230,576 clamp-delta entries, Model B 27,437).

**attn_v.** Keeping attn_v in Q4_K, as IQ2_XXS does, costs 0.60% in perplexity (19.22 against 19.10) and leaves the file 5.0 MB smaller than IQ2_XXS. Q4_K would buy generation speed (up to 1.04–1.09$\times$ the shipped speed, calculated, not measured: the native loader requires all 252 linears to be in the format). The re-encoding of attn_v at 2.0625 bits per weight (b) was killed and not rerun.

## Quality reference Model A: full text

**What it is.** Model A is our own slow reference: an entropy-coded 8-state trellis-coded quantizer with LDLQ rounding and rANS coding. It is not a llama.cpp format. It was evaluated by decoding it to an 8-bit (Q8_0) carrier of the decoded weights and running that carrier; the rate quoted for it is that of the entropy-coded stream, not of the carrier.

**Against the third-party LLVQ checkpoint.** Model A reaches perplexity 15.4768 (ratio 1.2424) at 2.128 bits per weight on the linears (2.47 over all tensors, a 1,240.5 MB file), against 1.371 at 2.1595 for the third-party LLVQ checkpoint of Table 17 (17.0722 in the HF fp32 harness, over that harness's fp16 12.4539). It is 0.098 nats per token better (paired $\Delta$NLL $-0.098 \pm 0.010$, 95% interval, standard error 0.005; better in 72 of 73 windows) at a rate 0.032 bits per weight (1.5%) lower. The two were run in different harnesses: Model A on its 8-bit carrier in llama.cpp and the checkpoint in the HF fp32 harness. On the checkpoint's own 8-bit carrier in llama.cpp it scores 17.1905 (1.380 over llama.cpp's fp16 12.4567), which gives $\Delta$NLL $-0.105$ and does not change the sign. This is a measured quality-versus-rate point against a third-party checkpoint, not against the LLVQ paper, whose 1.252 at 2.000 nominal bits per weight we did not reproduce.

**Why it is not deployable.** Model A runs from an 8-bit carrier at 0.59$\times$ the generation speed of IQ2_XXS, with 4076 MiB of weights in VRAM against 1183 MiB, and has no KL divergence in this paper. The gap between Model A and Model B is the price of a fixed-index, 256-entry, table-lookup format. Its state labels were also used for the one ablation that needs an entropy-coded model: labelling the trellis by the guru parity of the ganas instead of by Ungerboeck's set partitioning costs 7.7% in perplexity (Table 18, last block).

## Reproducibility details: notation table

**Notation.**

| Symbol | Meaning |
|---|---|
| $W \in \mathbb{R}^{m \times n}$ | linear weight after prescale and rotation ($m$ rows, $n$ columns) |
| $H = \frac{1}{N} X^\top X$ | layer Hessian from $N$ calibration tokens |
| $x_t \in \mathbb{R}^4$, $T = n/4$ | 4-dimensional symbols along a row |
| $u_t \in \{0, 1\}$ | trellis input bit at step $t$ |
| $s_t = (u_{t-2}, u_{t-1}, u_t) \in \mathcal{G}$ | trellis state, one of 8 (a gana) |
| $A$, $B = A + c$ | codebook of 256 int8 4-vectors and its coset |
| $z_0(s) = s^{(1)}$, $z_1 = u_t \oplus s^{(0)} \oplus s^{(2)}$ | subset bit ($A$ or $B$), stored index MSB |
| $\sigma$ | fp16 scale per 256 weights per row |
| $J_t(s)$ | Viterbi cost of the best path ending in state $s$ at step $t$ |
| $b_k$, $D_j(k)$, $\lambda_j$ | class rate (bpw), distortion of unit $j$ at class $k$, sensitivity |

# 1 Scalability of the format: supplementary derivations, fits and tables

**Scope.** The main text builds and measures B4 on Qwen3-4B, Llama-3.1-8B and Qwen3-14B; most measurements below are from Qwen3-4B. This appendix asks what the format implies for larger models. It contains propositions that are provable from the file layout and the code (Propositions 1–4), calculated tables with their assumptions, and a statement of what is not guaranteed. Apart from the Llama-3.1-8B builds (Appendix F) and the Qwen3-14B build (Section 4.3), which are used here only for the measured encoding time and the quality curve below, we have not encoded, loaded or run any model above 4B parameters. Each number is tagged MEASURED (from a log), CALC (arithmetic on stated assumptions, by a script), or PUBLISHED (copied from the table of the cited paper); the logs and scripts are in the supplementary material, with a number-to-log map.

**What this appendix does not show.** (1) *Quality at larger $N$ is not proven.* Storage and per-weight decode work are scale-free (Propositions 1, 3); the perplexity ratio to fp16 is a property of the weights and the encoder, and we measured it at 4B only. Published 2-bit results are mixed across scale (Table 13): for Llama-2 the ratio improves from 7B to 13B and then worsens at 70B, and for Llama-3 it is worse at 70B than at 8B. (2) *The generation-speed formula is an upper bound, not a prediction.* On Qwen3-4B the measured 201.7 tokens/s is 37% of the bandwidth bound (Table 22); we have no speed measurement above 4B. (3) *Encoding cost is not strictly linear in $N$*: the search is, the Hessian-aware part is not, and our extrapolation fits are indicative (Section 1.3). (4) *Hardware and speed losses stand:* CUDA on one laptop GPU only (no Metal, no Vulkan); prompt processing 0.69–0.74× IQ2_XXS (B4: 0.74×); batched decoding of 2 to 8 sequences 0.18–0.34× on the shipped path (0.92–1.09× with a post hoc multi-column kernel); CPU generation 0.56–0.62×; prompt energy 2.05× at depth 0 (Section 6). (5) *The impact tables are not iso-quality.* A 2-bit file is smaller than a 4-bit or 8-bit file and also has higher perplexity (on Qwen3-4B 1.336 times fp16 for B4 against 1.003 for Q8_0; no Q4_K_M row was measured); we claim a size and memory-traffic reduction at a stated quality loss, not equal quality. (6) We did not evaluate safety behaviour of quantized models.

## 1.1 Notation

A linear tensor is $W \in \mathbb{R}^{m\times n}$ ($m$ output rows, a multiple of 32; $n$ input columns, a multiple of 256) with $mn$ weights. A model has $N_{\text{lin}}$ linear weights in the decoder layers and $N_{\text{head}}$ weights in the output head. A symbol is 4 consecutive weights of a row; the trellis has $S = 8$ states and is reset every 32 symbols; the table has 256 entries. A tile is 32 rows × 256 columns and carries one rate class $k \in \{0,\dots,7\}$ with $r_k = 0, 1, 1.5, 2, 2.5, 3, 4, 6$ bits per weight, so a 256-weight row-block of class $k$ holds $\beta_k = 32r_k = 0, 32, 48, 64, 80, 96, 128, 192$ payload bytes. $b_{\text{lin}}$ is stored bits per linear weight, the paper's "linear bpw". Qwen3-4B has $N_{\text{lin}} = 3{,}633{,}315{,}840$ (252 tensors).

## 1.2 Storage

**Proposition 1 (Storage)** *Let a tensor with $m$ rows and $n$ columns have payload $P = \sum_{\text{row-blocks}} \beta_k$ bytes. Its native-file size is exactly*

$$B(m,n) = P + \underbrace{2n}_{\text{column prescale}} + \underbrace{2m}_{\text{row permutation}} + \underbrace{\tfrac{mn}{128}}_{\text{fp16 scale per 256}} + \underbrace{\tfrac{mn}{2048}}_{\text{u32 per tile}} + \underbrace{2048+16}_{\text{tables, header}} \quad \textit{bytes}, \tag{3}$$

*plus 8 bytes per clamped outlier. With $b_{\text{pay}} = 8P/(mn) \in [0,6]$ (the mean class rate) and $\rho$ the outlier fraction, the stored bits per weight are*

$$b = b_{\text{pay}} + \tfrac{17}{256} + \tfrac{16}{m} + \tfrac{16}{n} + \tfrac{16512}{mn} + 64\rho. \tag{4}$$

*Consequently (i) no term of the format grows with the number of weights; (ii) every term other than $b_{\text{pay}}$ and the constant $17/256 = 0.0664$ is $O(1/m + 1/n)$ and is at most $0.0151$ bits per weight for $m, n \geq 2560$ (below $0.0013$ at the Llama-3.1-405B shape $53{,}248 \times 16{,}384$); (iii) $b_{\text{pay}}$ is chosen by the encoder to meet a byte budget, so $b_{\text{lin}}$ is a design input and not a function of $N$.*

*Proof.* Count the stored fields. A row-block of class $k$ stores $\beta_k$ bytes of indices (64 symbols of $W_1 + W_2 + W_3$ bits). Each row-block has one fp16 scale (2 bytes per 256 weights, $mn/128$ bytes). Each tile has one 32-bit word, $4 \cdot \frac{m}{32} \cdot \frac{n}{256} = mn/2048$ bytes. Each input column has an fp16 prescale ($2n$), each row an int16 permutation index ($2m$), each tensor 2064 bytes of tables and header words (two 256-entry tables of 4 int8 values, 2048 bytes, and a 16-byte header; the check below measures the sum 2064). A clamped outlier stores an int16 row, an int16 column and an fp32 delta (8 bytes). Multiplying by $8/(mn)$ gives the second display, with $8 \cdot \frac{1}{128} + 8 \cdot \frac{1}{2048} = \frac{1}{16} + \frac{1}{256} = \frac{17}{256}$. □

**Check against the shipped file (MEASURED sizes, CALC formula).** We read the tensor sizes of the two native GGUF shards (headers only; script and log in the supplementary material) and the class histograms of the build log. For each of the 252 tensors, the size of its kernel tensor is exactly its payload plus $mn/128 + mn/2048$ plus 2064 bytes (residual 2064 in 252 of 252 tensors), its permutation tensor is $2m$ bytes and its prescale tensor $2n$ bytes (no mismatch), and the two clamp-delta tensors hold 4 bytes each per outlier (109,748 each for 27,437 outliers). The sum is 967,403,816 bytes, the size of the GGUF tensor tables, with difference 0; with the permutation packed to $\lceil \log_2 m \rceil$ bits and no tables the same formula gives the logged container blobs, 966,271,936 bytes, also with difference 0 (Table 21). The split of the 2064 bytes into 2048 (tables) and 16 (header) follows the layout comment of the native-file writer (supplementary material) and was not parsed separately. The measured $b_{\text{lin}} = 2.1301$ is 2.0529 (payload) + 0.0664 (scales and tile words) + 0.0107 (the $O(1/m + 1/n)$ terms, tables and clamp delta).

**Compression factor (Corollary).** For linear tensors the factor against fp16 is $16/b_{\text{lin}}$ and against Q8_0 (34 bytes per 32 weights, 8.5 bits per weight) is $8.5/b_{\text{lin}}$. At the measured $b_{\text{lin}} = 2.1301$ these are $7.51\times$ and $3.99\times$ (CALC). Holding the payload rate and outlier fraction of Qwen3-4B fixed and letting $m, n \to \infty$, $b_{\text{lin}} \to 2.0529 + 0.0664 + 0.0005 = 2.1198$ and the factors tend to $7.55\times$ and $4.01\times$ (CALC; this limit assumes the same class mix and outlier rate at every size, which we have not tested). Whole-file factors are smaller because the output head is kept at Q5_K: Qwen3-4B is $8{,}051{,}285{,}408/1{,}241{,}608{,}288 = 6.48\times$ against fp16 and $3.45\times$ against Q8_0 (CALC, MEASURED file sizes); Table 9 gives 6.7–7.4× for 8B to 405B.

**Layout limits (not tested above 4B).** The permutation index is 16 bits, so $m \le 65{,}535$; the kernel reads it as a signed short in the CUDA patch, which would need an unsigned read for $m > 32{,}767$, the case of the 405B gate and up tensors ($m = 53{,}248$). The payload offset field has 29 bits of 4-byte words (2 GiB per tensor), above the largest tensor in Table 9 (872M weights, at most 654 MB at class 6). All hidden and feed-forward widths of the models in Table 4 are multiples of 256 and all row counts multiples of 32.

## 1.3 Encoding cost

**Proposition 2 (Encoding cost)** *Fix a tensor $W \in \mathbb{R}^{m\times n}$, its Hessian factor $U$ and its class assignment.*

- *(a) The trellis search of one row over one 128-column slice (32 steps) performs $S\,c_k$ four-dimensional distance evaluations per step, where $c_k = 2^{W_1} + [W_2 > 0]\,2\min(4, 2^{W_1-1})2^{W_2} + [W_3 > 0]\,2\cdot 4\cdot 4\cdot 2^{W_3}$ counts the stage-1 candidates, the stage-2 refinements of the 4 best per half (classes 4–7) and the stage-3 refinements (class 7). Per weight this is $e_k = S\,c_k/4$ evaluations, with $e_k = 0, 32, 128, 512, 576, 768, 4608, 20992$ for $k = 0, \ldots, 7$ ($S = 8$). For stage-1-only classes with an 8-bit index (class 3, the class of 64% of the row-blocks of Qwen3-4B) this is 8 states × 2 branches × 128 candidates$/4 = 512$ evaluations per weight, because the index MSB is fixed by the branch.*
- *(b) The search needs $O(S \cdot 32)$ back-pointer state per row, independent of $m$ and $n$, because the trellis state is reset every 32 symbols (the same reset the decoder uses).*
- *(c) Given $U$ and the classes, rows are independent: the search is a parallel loop over $m$.*
- *(d) The search work for a model is $N_{\text{lin}} \sum_k p_k e_k$ evaluations, with $p_k$ the class mix; it is linear in $N_{\text{lin}}$ at fixed mix, and depends on the mix because the refinement stages of classes 4–7 multiply the work.*
- *(e) The Hessian-aware part is not linear in $N$: the error feedback to later columns costs $O(mn^2)$ multiply-adds per tensor ($n/2$ per weight), and each tensor re-factorizes its Hessian ($O(n^3)$, in double precision). With $N \propto Ld^2$ and $n \propto d$ the total is $O(Ld^3)$, not $O(Ld^2)$.*

*Proof.* By inspection of `best_cw`, `vit_k`, `encode` and `k_tensor` in the encoder script (supplementary material). `best_cw` evaluates all $2^{W_1}$ stage-1 entries (two halves of $2^{W_1-1}$), keeps the best TOP = 4 per half, evaluates $2\cdot 4\cdot 2^{W_2}$ stage-2 entries and, for class 7, $2\cdot 4\cdot 4\cdot 2^{W_3}$ stage-3 entries; `vit_k` repeats this for each of the $S$ states at each of 32 steps and keeps arrays of length $S \times 32$ per row. `encode` loops over 128-column blocks, calls `vit_k` on all rows of a class at once, and then applies `W[:, i1:] -= Eb @ Uc[i0:i1, i1:]`, which costs $m \cdot 128 \cdot (n - i_1)$ multiply-adds per block and $\approx mn^2/2$ in total. `k_tensor` computes a Cholesky factorization, an inverse and a second factorization of the $n \times n$ Hessian for every tensor, in float64 (on the GPU for $n \le 10{,}000$, otherwise on the host); q, k and v share one Hessian and gate and up share another, but each tensor repeats the factorization. □

**Remark on the state count.** The search term is linear in $S$: a 256-state code would multiply every $e_k$ by 32. The "within 0.21 dB of the distortion-rate bound" figure of Marcellin & Fischer (1990) quoted in the background section of the main text is for a 256-state code on a memoryless uniform source; it is not a property of our 8-state code, and we make no claim of that kind (16 states already lower perplexity by 1.7% against 8).

**Measured constants (Qwen3-4B, one RTX 5070 Ti Laptop GPU, 16 CPU threads).** The build script (log in the supplementary material) took 4623 s end to end for 252 tensors and 3,633,315,840 weights, i.e. $1.272$ $\mu$s per weight (MEASURED, CALC division). That wall time includes the calibration forward pass and Hessian accumulation, comparison metrics against Model A and packing. The sum of the per-tensor quantizer times (`t_q`, which wraps `k_tensor` including its factorizations) is 2768.3 s, $0.762$ $\mu$s per weight, 59.9% of the wall time. The per-layer wall time was not constant: 170–278 s for layers 0–3 and $87.5$ s on average for layers 23–35 ($0.867$ $\mu$s per weight); the log does not record whether other jobs used the GPU, so we do not treat any one value as the constant. Peak GPU memory was 9.36 GB (`max_memory_allocated`). Per tensor type the quantizer time per weight was 0.45 (ffn_up), 0.53 (ffn_gate), 1.04 (ffn_down, $n = 9728$), 0.69 (attn_q), 0.74 (attn_output), 1.32 (attn_k) and 3.08 (attn_v, classes 6–7) $\mu$s.

**Is the cost linear? (indicative fits).** Evaluations per weight differ by class, and the three feed-forward tensors have about the same number of weights but ffn_down has $n = 9728$ against 2560: its quantizer time per weight is 2.3 times that of ffn_up while its evaluation count is only 1.3 times larger (CALC from the class histograms). We fitted the per-tensor `t_q` by non-negative least squares to $a \cdot E_t + b \cdot g_t$, where $E_t$ is the number of distance evaluations of tensor $t$ implied by its histogram and $g_t = mn^2$ (fit A) or $n^3$ (fit B). A one-term model on $E_t$ alone explains none of the variation on layers $\geq 8$ ($R^2 = -0.12$). Fit A gives $R^2 = 0.87$ (all 252 tensors; 0.81 on layers $\geq 8$) with $a = 0.48$ s per $10^9$ evaluations and $b = 72$ s per $10^{12}$ of $mn^2$, and attributes 56% of the fitted time to evaluations and 44% to the $n$-dependent term; fit B gives $R^2 = 0.81$ (0.69 on layers $\geq 8$) with a 27% $n^3$ share. The coefficient $b$ corresponds to 14 GFLOP/s of the feedback product, far below the nominal FP32 rate of a GPU of this class (order 10 TFLOP/s), so the term is not that product alone; the mechanism (candidates: the double-precision factorizations, host–device traffic, per-block launch overhead) is not isolated, and the fits are contaminated by the unrecorded GPU sharing. They show that the 4B log is not consistent with time proportional to weights alone; they are not a validated model.

Table 4: Encoding time on one RTX 5070 Ti Laptop-class GPU, in GPU-hours (CALC; extrapolation from the Qwen3-4B log; †: the Qwen3-14B build, MEASURED afterwards, 10,851 s with GPU memory capped at 0.58 of 12 GB and the 17,408-dimension factorizations on the CPU). "Linear" multiplies $N_{\text{lin}}$ by the measured $0.762$ $\mu$s (quantizer only) or $1.272$ $\mu$s (whole script) per weight. Fits A and B apply the fitted coefficients above to the tensor shapes of each model, with the class mix of Qwen3-4B (865 evaluations per weight); the 4B row returns $0.75$ and $0.69$ against the measured $0.77$ (quantizer) and $1.28$ (whole script). The last column is the formula GPU memory (calibration activations $544 \times 2048 \times d$ fp16 plus three $n^2_{\max}$ fp32 Hessians plus the largest fp32 matrix) divided by 0.74, the ratio of that formula to the 9.36 GB measured at 4B. For $n > 10{,}000$ the code factorizes on the host; this was not timed separately and is included only in the measured Qwen3-14B value (†). Multiply by a cloud price per GPU-hour assuming the GPU is at least as fast as the laptop part (untested); at USD 0.89–4.29 per hour (Table 8) the 70B linear estimate is USD 13–104, the 405B USD 76–609, and fit A for 405B USD 232–1117.

| Model | $N_{\text{lin}}$ (B) | linear, quantizer | linear, whole script | fit A | fit B | GPU memory (GB) |
|---|---|---|---|---|---|---|
| Qwen3-4B | 3.63 | 0.77* | 1.28* | 0.75 | 0.69 | 9 |
| Llama-3.1-8B | 6.98 | 1.48 | 2.47 | 1.77 | 1.48 | 16 |
| Qwen3-14B | 13.21 | 2.80 | 4.67 (3.01†) | 3.77 | 3.00 | 21 |
| Qwen3-32B | 31.21 | 6.60 | 11.0 | 10.4 | 9.85 | 27 |
| Llama-3.1-70B | 68.45 | 14.5 | 24.2 | 26.9 | 19.8 | 39 |
| Llama-3.1-405B | 401.65 | 85.0 | 142.0 | 260.4 | 162.8 | 100 |

* MEASURED (build log: 2768.3 s quantizer, 4623 s whole script); † MEASURED (caption); all other cells CALC.

**Consequences.** (i) Encoding is a one-off offline cost: 14–27 GPU-hours at 70B and 85–260 GPU-hours (3.5–11 GPU-days) at 405B across the columns of Table 4. It is not a bottleneck for deployment, but it is not free of super-linear terms, and the 405B row exceeds the memory of one 80 GB GPU (and the host would need about 29 GB for the Hessians, CALC). (ii) Savings that would lower these numbers were not built: factorizing once per shared input (4 instead of 7 per layer), or restricting the feedback to a window of columns (which changes the quantizer and its quality).

(iii) Unlike a single-stage trellis, whose search cost does not depend on the rate, the cost here depends on the class mix: a class-6 tile costs 9 times and a class-7 tile 41 times a class-3 tile (4608 and 20992 against 512 evaluations per weight), which is why attn_v (79% class 6, 21% class 7) took 3.08 $\mu$s per weight, 3 to 7 times the feed-forward tensors.

## 1.4 Decode work and generation speed

**Proposition 3 (Decode work per weight)** *In the batch-1 activation-table GEMV of Section 3:*

*(a) a 4-weight symbol of class $k$ costs $\rho_k = 0, 1, 1, 1, 2, 2, 2, 3$ table reads and adds plus a constant amount of integer work per symbol for the trellis state, index and sign (measured on sm_120: 278 SASS instructions per 32 class-3 symbols, 48 of them shared-memory loads, i.e. 8.7 per symbol; instruction counts did not predict tg128 in our measurements); the per-slice table of 256 entries for each of 32 four-activation chunks (32 KiB) is built once per thread block and 128-activation slice, costing $256/R$ multiply-adds per weight for $R$ rows per block. None of $\rho_k$, 32 KiB or $R$ depends on $m$, $n$ or $N$.*

*(b) A generated token reads $B_{\mathrm{tok}} = (b_{\mathrm{lin}} N_{\mathrm{lin}} + b_{\mathrm{head}} N_{\mathrm{head}})/8$ bytes of weights (plus the KV cache), and performs $N_{\mathrm{lin}}\bar{\rho}/4$ lookups, $\bar{\rho} = \sum_k p_k \rho_k$ (1.147 for Qwen3-4B, CALC; the paper's range is 1.06–1.20).*

*(c) If the weights do not fit in cache (L2 is 36 MiB; every model here has $\geq 1.2$ GB of weights), then*

$$\textit{tokens per second} \;\leq\; \frac{\mathrm{BW}}{B_{\mathrm{tok}}}. \tag{5}$$

*This is an* upper bound*; reaching it requires the kernel to saturate the memory interface.*

*Proof.* (a) is the kernel description (Section 3) and the class table of Section 3; (b) follows from Proposition 1 and the output-head format (Q5_K, 5.5 bits per weight in both our file and IQ2_XXS); (c) every weight byte must cross the memory interface once per token. □

Both the byte count and the lookup count are proportional to $N$, so under either limiter the time per token is linear in $N$, and the ratio of our speed to a baseline format at the same $N$ is governed by per-weight constants (bytes, lookups), not by $N$. That ratio is scale-free only if both kernels stay in the same regime (bandwidth-bound or latency-bound) as $N$ grows; we have not measured this.

**Where the bound is tight, and where it is not (MEASURED anchor).** On Qwen3-4B, Model B reads $B_{\mathrm{tok}} = 967{,}403{,}816 + 267{,}407{,}360 = 1{,}234{,}811{,}176$ bytes per token (linear tensors plus the tied head, 388,956,160 weights at 5.5 bits), and generates 201.7 tokens/s, an effective 249 GB/s, 0.371 of the nominal 672 GB/s of the part (Table 22). On the same GPU the Q8_0 file reaches 513 GB/s (0.763 of nominal). So at 4B generation with the 2-bit formats is limited by the kernel (decode work and latency over small matrices; not profiled), not by memory bandwidth (cf. the roofline view of Yuan et al., 2024), and the bound of Table 5 is loose by about a factor 2.7 at the nominal bandwidth. If this efficiency carried over unchanged, speeds would be 0.37 of the bounds; whether it does is untested, and may be better (larger matrices amortize launch latency) or worse (larger activation tables, more cache pressure). The byte model also over-predicts the effect of context: at depth 4096 the Qwen3-4B KV cache adds 0.604 GB to the 1.235 GB of weights, so bytes alone would predict $0.67\times$ the depth-0 speed, whereas the measured ratio is $159.7/201.7 = 0.79$ (CALC).

**Known issue: perplexity on the token-generation path.** Perplexity and KL divergence in this paper use the batched path (the 8-bit slab, Section 3); generation uses the GEMV. A check run with micro-batch 1 (llama-perplexity `-ub 1`, so every token goes through the GEMV) on the first 8 WikiText-2 windows of context 4096 (against the batched-path runs; MEASURED, logs in the supplementary material) gives a cumulative perplexity over the 8 windows of 18.2500 against 18.3234 for Model B ($-0.40\%$) and 17.3668 against 17.1477 for B3 ($+1.28\%$). Per window, the difference in mean NLL (token path minus batched path, nats per token) is $-0.006$, $-0.003$, $+0.004$, $-0.005$, $-0.006$, $-0.007$, $-0.002$, $-0.006$ for Model B and $+0.009$, $-0.002$, $0.000$, $+0.080$, $+0.006$, $+0.006$, $-0.001$, $+0.004$ for B3 (CALC from the running perplexities): B3's excess comes almost entirely from window 4 (perplexity 11.29 against 10.42), and its other seven windows average $+0.003$. The IQ2_XXS control (stock mmvq against mmq) moves by $+0.03\%$ over the same 8 windows. Cause, localized (12 debugging runs, supplementary material): like every window after the first, window 4 starts with an ordinary token at position 0 and no BOS (llama-perplexity's strided mode inserts the BOS after decoding), so that token becomes the attention sink; of the 8 windows, only window 4 moves much. Every kernel that

uses B3's exact decoded weights handles this start badly: window-4 perplexity is 11.29 on the shipped GEMV and 17.9–20.95 on the other exact paths (two-pass 15-bit activations, the VR-domain MMQ), against 10.42 on the batched 8-bit slab path, 7.35 for fp16 and 11.15 for Model B. Switching only the seven tensors blk.0–6 ffn_gate to the slab path restores it (10.39); they hold 148,300 of B3's 233,221 clamp-delta entries. B3+ shows the same effect ($+0.195$ nats on window 4 at micro-batch 1). The kernels are exact (GEMV against a bit-level decode: max relative error $2.9 \times 10^{-6}$ on all 252 tensors); the weakness is in these encoded tensors, and re-encoding them is future work. B3's and B3+'s perplexities therefore describe the batched path; on the generation path B3 measures $+1.28\%$ over these 8 windows, and Model B and IQ2_XXS show no gap.

Table 5: Upper bound on batch-1 tokens per second at depth 0, $\mathrm{BW}/B_{\mathrm{tok}}$ (CALC; not a prediction: Table 22 shows 37% attained at 4B for the 2-bit formats and 76% for Q8_0). $B_{\mathrm{tok}}$ is the weights read per token (GB, $10^9$ bytes). n/f: the file plus 10% headroom does not fit in the memory of one such GPU (12, 24 and 80 GiB). Format assumptions are in Table 8; Llama-3.1 shapes are in Table 8. Bandwidths: 672 (laptop part), 1008 (RTX 4090), 3350 GB/s (H100 SXM).

| Model | format | $B_{\mathrm{tok}}$ (GB) | 672 GB/s | 1008 GB/s | 3350 GB/s |
|---|---|---|---|---|---|
| Llama-3.1-8B | fp16 | 15.01 | n/f | 67 | 223 |
| Llama-3.1-8B | Q4_K_M | 4.55 | 148 | 222 | 736 |
| Llama-3.1-8B | IQ2_XXS | 2.23 | 301 | 452 | 1503 |
| Llama-3.1-8B | ours | 2.22 | 303 | 454 | 1509 |
| Llama-3.1-70B | fp16 | 139.00 | n/f | n/f | n/f |
| Llama-3.1-70B | Q4_K_M | 42.14 | n/f | n/f | 80 |
| Llama-3.1-70B | IQ2_XXS | 19.04 | n/f | 53 | 176 |
| Llama-3.1-70B | ours | 18.95 | n/f | 53 | 177 |
| Llama-3.1-405B | fp16 | 807.50 | n/f | n/f | n/f |
| Llama-3.1-405B | Q4_K_M | 244.77 | n/f | n/f | n/f |
| Llama-3.1-405B | IQ2_XXS | 108.94 | n/f | n/f | n/f |
| Llama-3.1-405B | ours | 108.39 | n/f | n/f | n/f |

**Reading Table 5.** At the same bandwidth the bound for our file is 6.8–7.5× that of fp16 and 2.0–2.3× that of Q4_K_M (ratio of bytes read), and equal to IQ2_XXS to within 1% (our 2.1301 against its 2.141 bits per weight). The speed ratio to IQ2_XXS (Model B: 0.976 [0.918, 1.033]× at 4B, Table 9) is therefore not a bandwidth effect and we make no claim that it holds at other scales. The ratios to fp16 and Q4_K_M shrink with context because the KV cache is not compressed (Table 6); the file-size and memory ratios of Table 9 do not.

Table 6: Ratio of bytes read per token, weights plus fp16 KV cache at the stated depth (CALC; Llama-3.1 with 8 KV heads of dimension 128).

| Model | depth (tokens) | KV read per token (GB) | fp16 / ours | Q4_K_M / ours |
|---|---|---|---|---|
| Llama-3.1-8B | 0 | 0.00 | 6.76 | 2.05 |
| Llama-3.1-8B | 4,096 | 0.54 | 5.64 | 1.85 |
| Llama-3.1-8B | 32,768 | 4.29 | 2.96 | 1.36 |
| Llama-3.1-70B | 0 | 0.00 | 7.34 | 2.22 |
| Llama-3.1-70B | 4,096 | 1.34 | 6.92 | 2.14 |
| Llama-3.1-70B | 32,768 | 10.74 | 5.04 | 1.78 |

## 1.5 Fixed runtime overhead

**Proposition 4 (Prompt-path overhead)** *The prompt-path pre-pass of Section 3 writes one weight matrix at a time into an 8-bit slab of* $34\,mn/32$ *bytes (Q8_0 layout), allocated per matrix multiplication from a pool that reuses buffers. The slab is therefore bounded by the* largest *single matrix and does not depend on the number of layers* $L$ *or of tensors. Token generation does not use it. Relative to the weights it is* $34 \max(mn)/32$ *over* $b_{\mathrm{lin}} N_{\mathrm{lin}}/8$*, which for the model shapes below is* $\approx (0.77\text{–}1.08)/L$*.*

*Proof.* The allocation is `rows*(K/32)*34` bytes in the PPC2 branch of the CUDA patch (one `ggml_cuda_pool_alloc` per call), and generation uses the native GEMV. The ratio is arithmetic (Table 7). □

The overhead is not $O(1)$ in the width of the model (the largest matrix grows as $d \cdot d_{\text{ff}}$), and its share falls with depth, not with the parameter count as such: from 2.1% at 4B (36 layers) to 0.86% at 405B (126 layers), a factor 2.5 for a factor 110 in $N_{\text{lin}}$. Only the slab term has a derivation. The measured peak-memory excess of the native file over IQ2_XXS is $+56$ MiB during prompt processing (slab 25.2 MiB; the remaining 30.8 MiB are not decomposed) and $+38$ MiB during generation (2922 against 2884 MiB, where no slab exists; CALC); we make no statement about how those non-slab parts scale.

Table 7: Slab size against Model B weight bytes (CALC; Llama-3.1 shapes from Table 8). Qwen3-4B: measured $+56$ MiB at depth 0.

| Model | layers | largest matrix (M weights) | slab (MB) | ours weights (GB) | slab / weights |
|---|---|---|---|---|---|
| Qwen3-4B | 36 | 24.9 | 26.5 | 1.23 | 2.14% |
| Llama-3.1-8B | 32 | 58.7 | 62.4 | 2.22 | 2.81% |
| Llama-3.1-70B | 80 | 234.9 | 249.6 | 18.95 | 1.32% |
| Llama-3.1-405B | 126 | 872.4 | 926.9 | 108.39 | 0.86% |

**Table size.** The decoder's constant state is the two 256-entry tables of 4 int8 values (2048 bytes), the coset vector and the class shifts, 2073 bytes in the container and 2048 bytes reserved per tensor in the GGUF layout; with 252 tensors this is 516,096 bytes, 0.042% of the file. The per-thread-block activation table is $256 \times 32 \times 4 = 32{,}768$ bytes. All of these are fixed by the format (256 entries, 32 chunks) and are independent of $N$: the format never grows its codebook with the model, which is also why it cannot adapt it.

## 1.6 Impact at larger scale (CALC with stated assumptions)

Tables 8–12 are calculated from the formats of Proposition 1 under the assumptions listed first; none of them is measured, and none assumes equal quality across formats.

Table 8: Assumptions behind the impact tables. Prices were read from the cited pages on 2026-10-17 and change; they are ranges, not forecasts.

**Shapes.** Llama-3.1 8B/70B/405B: 32/80/126 layers, model dim 4096/8192/16384, FFN dim 14336/28672/53248, 8 KV heads (Llama 3 report, Table 3, `https://arxiv.org/abs/2407.21783`); vocabulary 128,256 from the 8B config (`https://huggingface.co/unsloth/Meta-Llama-3.1-8B`), assumed equal for 70B and 405B; untied embedding and head. Qwen3-14B and 32B: Hugging Face config.json of Qwen/Qwen3-14B and Qwen/Qwen3-32B.

**Bits per weight.** fp16 16; Q8_0 8.5; Q4_K_M 4.85 for all weights (the nominal value; llama.cpp lists 4.8944 for the whole 8B file, `https://github.com/ggml-org/llama.cpp/blob/master/tools/quantize/README.md`, and our uniform value is 0.9% below its 4.58 GiB, CALC); IQ2_XXS 2.141 on linear tensors (MEASURED on Qwen3-4B) and ours 2.1301 (MEASURED); head Q5_K (5.5) and untied input embedding Q2_K (2.625) for both 2-bit files; 6 MB metadata. Cross-check: the same recipe gives 2.407 GB for Llama-3.1-8B IQ2_XXS against a measured local file of 2.399 GB (+0.34%) and 8.538 GB for Q8_0 against 8.541 GB (–0.03%). Sensitivity: ours at 2.05 or 2.30 changes the 70B file by –3.6% or +7.5%.

**KV cache.** fp16, 8192 tokens, 1 or 32 sequences; $2 \cdot L \cdot 8 \cdot 128 \cdot 2$ bytes per token (131,072 / 327,680 / 516,096 B for 8B / 70B / 405B).

**GPU fit.** capacity 24 or 80 GiB, 90% usable, 1 GiB reserved, weights and KV split without replication; 70B at one 24 GiB card leaves 0.2% (IQ2_XXS) or 0.6% (ours) of the usable memory. Bandwidth: RTX 5070 Ti Laptop 672 GB/s (12 GB GDDR7, 192 bit; Notebookcheck lists 677, `https://www.notebookcheck.net/Nvidia-GeForce-RTX-5070-Ti-Laptop-Benchmarks-and-Specs.934945.0.html`), RTX 4090 1008 GB/s (384 bit $\times$ 21 Gbit/s; 24 GB, `https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/`), H100 SXM 3.35 TB/s, 80 GB (`https://www.nvidia.com/en-us/data-center/h100/`).

**Price per GPU-hour (USD).** 24 GiB class: 0.89 (RTX 4090, RunPod secure cloud, `https://www.runpod.io/pricing`) to 1.29 (A10 24 GB, Lambda, `https://lambda.ai/pricing`); 80 GiB class: 2.89 (H100 PCIe, RunPod) to 4.29 (1$\times$ H100 SXM, Lambda). The classes differ in bandwidth and speed; costs below are per replica-hour, not per token.

**Egress.** USD 0.09 per GB, the AWS internet-egress list rate of the first tier (`https://aws.amazon.com/vpc/pricing/`); hubs that serve downloads free, or large publishers with negotiated rates, pay less, so this is an upper bound.

**Reading the impact tables.** (i) *Access.* On a 12 GiB laptop GPU an 8B model needs 2.4 GB in our format against 16.1 GB in fp16 (does not fit) and 4.9 GB in Q4_K_M; a 70B model fits one 24 GiB card at a single 8K sequence (0.6% margin; calculated, not run, and assuming the 1 GiB reserve covers all runtime buffers) and two 80 GiB cards at 32

Table 9: File size in GB ($10^9$ bytes), CALC under Table 8. The last columns are size ratios. IQ2_XXS and ours are within 0.5% of each other, as they are at 4B (1.004×, MEASURED file sizes); the difference between them is quality (Section 4), not size.

| Model | fp16 | Q8_0 | Q4_K_M | IQ2_XXS | ours | fp16 / ours | Q4_K_M / ours |
|---|---|---|---|---|---|---|---|
| Llama-3.1-8B | 16.1 | 8.5 | 4.9 | 2.4 | 2.4 | 6.70 | 2.03 |
| Llama-3.1-70B | 141.1 | 75.0 | 42.8 | 19.4 | 19.3 | 7.31 | 2.22 |
| Llama-3.1-405B | 811.7 | 431.2 | 246.1 | 109.6 | 109.1 | 7.44 | 2.26 |

Table 10: Minimum number of GPUs needed to hold weights, KV cache and a 1 GiB reserve (CALC). Left pair: 24 GiB GPUs; right pair: 80 GiB GPUs; "1 seq" = one 8192-token sequence, "32 seq" = 32 concurrent such sequences. Memory only: no interconnect, parallelism or throughput model. The † cells fit with under 1% of usable memory to spare (0.2% IQ2_XXS, 0.6% ours).

| Model | format | 24 GiB, 1 seq | 24 GiB, 32 seq | 80 GiB, 1 seq | 80 GiB, 32 seq |
|---|---|---|---|---|---|
| Llama-3.1-8B | fp16 | 1 | 3 | 1 | 1 |
| Llama-3.1-8B | Q4_K_M | 1 | 2 | 1 | 1 |
| Llama-3.1-8B | IQ2_XXS | 1 | 2 | 1 | 1 |
| Llama-3.1-8B | ours | 1 | 2 | 1 | 1 |
| Llama-3.1-70B | fp16 | 7 | 10 | 2 | 3 |
| Llama-3.1-70B | Q4_K_M | 3 | 6 | 1 | 2 |
| Llama-3.1-70B | IQ2_XXS | 1† | 5 | 1 | 2 |
| Llama-3.1-70B | ours | 1† | 5 | 1 | 2 |
| Llama-3.1-405B | fp16 | 36 | 41 | 11 | 13 |
| Llama-3.1-405B | Q4_K_M | 11 | 17 | 4 | 5 |
| Llama-3.1-405B | IQ2_XXS | 5 | 11 | 2 | 4 |
| Llama-3.1-405B | ours | 5 | 11 | 2 | 4 |

concurrent sequences (Table 10), against 7 and 3 for fp16. (ii) *Serving.* Fewer GPUs per replica is the effect that can be calculated; for 70B and 405B it lowers the memory-driven cost per replica-hour by 2.0 to 7.2× against fp16 and by 1.0 to 3.0× against Q4_K_M, depending on the GPU class (Table 11); for 8B nothing changes (one GPU in every format). The quality loss is not equated and throughput is not modelled. (iii) *Traffic.* The per-download saving is 13.7 GB (8B), 121.8 GB (70B) and 702.6 GB (405B) against fp16, and 2.5, 23.5 and 137.0 GB against Q4_K_M. (iv) *Energy proxy.* Bytes read per token fall to 0.134–0.148 of fp16 and 0.44–0.49 of Q4_K_M (Table 5). Bytes moved are a proxy for the memory-interface energy of generation only. The measured energy of our file on Qwen3-4B is 1.01× IQ2_XXS per generated token (0.502 against 0.497 J) but 2.05× per prompt token at depth 0 (median of 3 rounds, 1.70–4.10; 1.58× at depth 4096), because the prompt path decodes each matrix to 8 bits before the multiply; whole-workload energy against fp16 or Q4_K_M was not measured.

## 1.7 What is not guaranteed

**Quality at larger $N$.** Nothing above implies that the perplexity ratio to fp16 stays at 1.336 (B4), or improves, at 8B–405B. Propositions 1–4 concern bytes and operations; the ratio depends on the weight statistics, the Hessians, the 256-entry table at fixed rate (trained on Qwen3-4B) and the encoder. Published 2-bit results do not give a monotone trend either (Table 13, all PUBLISHED, ratios CALC):

- Llama-2 (QTIP Table 5 at ctx 4096; AQLM Table 1): the ratio improves from 7B to 13B for every method in both tables (QTIP 1.145 to 1.118; AQLM 1.287 to 1.225) and is *worse* at 70B than at 13B (QTIP 1.186; AQLM 1.263). The absolute perplexity gap shrinks with scale (QTIP 0.74, 0.54, 0.58) and so do the zero-shot drops (Table 14: AQLM average of five tasks 5.07, 4.06, 1.42 points).
- Llama-1 (QTIP, ctx 2048): the ratio is 1.148, 1.139, 1.178, 1.193 for 7B, 13B, 30B, 65B, i.e. it rises beyond 13B.
- Llama-3 and 3.1 (QTIP, ctx 8192): the ratio is much worse at larger sizes (base: 1.323 at 8B, 1.919 at 70B; instruct: 1.203, 1.443 and 1.935 at 8B, 70B, 405B, with the 405B baseline Meta's FP8 rather than BF16). The authors note that their 70B zero-shot results are affected by quantizing layer 0 v; the perplexity rows are as published.

Table 11: Cost of one replica holding one 8192-token sequence, USD per replica-hour (CALC from Table 10 and the prices of Table 8; ranges are the low and high price of each GPU class). "Any class" takes the cheaper of the 24 GiB and 80 GiB class. This is memory-driven cost and not a cost per token: it ignores that a 24 GiB card has lower bandwidth than an H100 and that 4-bit and 2-bit files differ in quality.

| Model | format | any class (USD/h) | 80 GiB only (USD/h) |
|---|---|---|---|
| Llama-3.1-8B | fp16 | 0.89–1.29 | 2.89–4.29 |
| Llama-3.1-8B | Q4_K_M | 0.89–1.29 | 2.89–4.29 |
| Llama-3.1-8B | ours | 0.89–1.29 | 2.89–4.29 |
| Llama-3.1-70B | fp16 | 5.78–8.58 | 5.78–8.58 |
| Llama-3.1-70B | Q4_K_M | 2.67–3.87 | 2.89–4.29 |
| Llama-3.1-70B | ours | 0.89–1.29 | 2.89–4.29 |
| Llama-3.1-405B | fp16 | 31.79–46.44 | 31.79–47.19 |
| Llama-3.1-405B | Q4_K_M | 9.79–14.19 | 11.56–17.16 |
| Llama-3.1-405B | ours | 4.45–6.45 | 5.78–8.58 |

Table 12: Storage and download traffic saved by shipping our file instead of the stated format (CALC). Per-download saving, total for $10^3$ and $10^6$ downloads, and the AWS list egress price of $10^6$ downloads at USD 0.09 per GB (upper bound). Against IQ2_XXS the saving is below 1% of the file.

| Model | versus | GB saved | TB per $10^3$ | PB per $10^6$ | egress, $10^6$ (USD M) |
|---|---|---|---|---|---|
| Llama-3.1-8B | fp16 | 13.7 | 13.7 | 13.67 | 1.23 |
| Llama-3.1-8B | Q4_K_M | 2.5 | 2.5 | 2.48 | 0.22 |
| Llama-3.1-70B | fp16 | 121.8 | 121.8 | 121.81 | 10.96 |
| Llama-3.1-70B | Q4_K_M | 23.5 | 23.5 | 23.48 | 2.11 |
| Llama-3.1-405B | fp16 | 702.6 | 702.6 | 702.62 | 63.24 |
| Llama-3.1-405B | Q4_K_M | 137.0 | 137.0 | 136.97 | 12.33 |

Kumar et al., "Scaling Laws for Precision" (arXiv 2411.04330, `https://arxiv.org/abs/2411.04330`) report, for models up to 1.7B parameters, that the degradation from post-training quantization grows with the number of training tokens. If tokens per parameter is the driver, the Qwen3-4B anchor and the Llama-3 rows are on the hard side, and older models such as Llama-2 are on the easy side; we did not test this. The claim that larger models quantize better is therefore true for some metrics and families (absolute gaps, zero-shot drops, Llama-2 7B→13B) and false for others (perplexity ratio, Llama-1 beyond 13B, Llama-3), and we do not rely on it.
On Qwen3-4B the excess negative log-likelihood per token of our format (the logarithm of the ratio) is 0.290 for B4 (0.427 for Model B), against 0.831 for IQ2_XXS (CALC; excess loss remaining = (ratio $-$ 1) / (IQ2_XXS ratio $-$ 1)). For comparison, the same quantity changes by a factor 1.5 across Llama-2 7B to 70B (QTIP: 0.135, 0.112, 0.170) and 2.3 across Llama-3 8B to 70B (0.280, 0.652). Three measured models in two families (Section 4.3) are a first curve, not a scale statement.

**Speed, memory and hardware.** (i) Generation: batch-1 decoding of B4 at 1.096 [1.034, 1.158]× IQ2_XXS at 4B (one exclusive session with all files; 1.109 [1.099, 1.119] in a B4-only session; B3 and Model B tie IQ2_XXS, Model B at 1.003× and 0.967× in earlier sessions); batched decoding of 2 to 8 sequences at 0.18–0.34× on the shipped path; the prompt path is 0.74× (Model B, earlier sessions: 0.70× at depth 0 and 0.75× at depth 4096), with +56 MiB peak memory and 2.05× prompt energy; no tuned kernel above 4B was run. (ii) CUDA only, measured on a single RTX 5070 Ti Laptop GPU; the x86 backend generates at 0.56–0.62× IQ2_XXS and is format-bound at about 0.7×; the phone estimates are unmeasured; there is no Metal or Vulkan kernel, so Apple-silicon and non-NVIDIA GPUs are not covered. (iii) The runtime is a patch set on one llama.cpp commit and is not merged upstream. (iv) Multi-GPU runs (needed for 70B in fp16 or 405B in any format here) were not tested with this runtime. (v) The 16-bit permutation and 29-bit offsets are layout limits that have only been exercised at 4B. (vi) Calibration memory grows with width (Table 4); at 405B the build as written does not fit one 80 GB GPU.

**What would settle the open points.** A 3-point WikiText-2 curve for the same format with the same tables now exists (Qwen3-4B, Llama-3.1-8B, Qwen3-14B; Table 1): on Qwen3, B4's margin over IQ2_XXS shrinks from 0.54 nats at 4B

Table 13: PUBLISHED 2-bit Wikitext2 perplexity across model size, arXiv v4 tables of Tseng et al. (2024) (QTIP Tables 5, 7 and 8; Table 5 is the HYB code with fine-tuning) and Egiazarian et al. (2024) (AQLM Table 1, no end-to-end fine-tuning); each cell is quantized / fp16 = ratio (ratio CALC, same table's fp16 or BF16 baseline). QTIP T5 "AQLM 2+" is AQLM as reported in QTIP's table at ctx 4096. Contexts differ by row; ratios are not comparable across rows.

| Family and source | method | quantized / fp16 = ratio, by size | | | |
|---|---|---|---|---|---|
| *Llama-2, QTIP T5 (Wikitext2, ctx 4096); sizes 7B, 13B, 70B; fp16/BF16 5.12, 4.57, 3.12* | | | | | |
| | QTIP 2.00 | 5.86/5.12=1.145 | 5.11/4.57=1.118 | 3.70/3.12=1.186 | |
| | QuIP# 2.00 | 6.19/5.12=1.209 | 5.35/4.57=1.171 | 3.91/3.12=1.253 | |
| | AQLM 2+ | 6.14/5.12=1.199 | 5.33/4.57=1.166 | 3.83/3.12=1.228 | |
| *Llama-2, AQLM T1 (Wikitext2, no e2e FT); sizes 7B, 13B, 70B; fp16/BF16 5.12, 4.57, 3.12* | | | | | |
| | AQLM 2.02/1.97/2.07 | 6.59/5.12=1.287 | 5.60/4.57=1.225 | 3.94/3.12=1.263 | |
| | QuIP# 2.02/2.01/2.01 | 8.22/5.12=1.605 | 6.06/4.57=1.326 | 4.16/3.12=1.333 | |
| *Llama-1, QTIP T5 (Wikitext2, ctx 2048); sizes 7B, 13B, 30B, 65B; fp16/BF16 5.68, 5.09, 4.1, 3.53* | | | | | |
| | QTIP 2.00 | 6.52/5.68=1.148 | 5.80/5.09=1.139 | 4.83/4.10=1.178 | 4.21/3.53=1.193 |
| | QuIP# 2.00 | 6.86/5.68=1.208 | 5.97/5.09=1.173 | 5.02/4.10=1.224 | 4.36/3.53=1.235 |
| *Llama-3, QTIP T7 (Wikitext2, ctx 8192); sizes 8B, 70B; fp16/BF16 5.54, 2.59* | | | | | |
| | QTIP 2.00 | 7.33/5.54=1.323 | 4.97/2.59=1.919 | | |
| | QuIP# 2.00 | 7.84/5.54=1.415 | 5.77/2.59=2.228 | | |
| *Llama-3.1, QTIP T8 (Wikitext2, ctx 8192, instruct); sizes 8B, 70B, 405B; fp16/BF16 6.5, 3.52, 1.7* | | | | | |
| | QTIP 2-bit | 7.82/6.50=1.203 | 5.08/3.52=1.443 | 3.29/1.70=1.935 | |

Table 14: PUBLISHED zero-shot accuracy drop (points; fp16 minus 2-bit) for Llama-2 7B/13B/70B: QTIP Table 6 (ArcC, ArcE, WinoGrande) and AQLM Table 1 (average of five tasks).

| Metric (2-bit) | 7B | 13B | 70B |
|---|---|---|---|
| QTIP 2-bit ArcC (T6) | 4.30 | 4.20 | 3.10 |
| QTIP 2-bit ArcE (T6) | 3.70 | 2.50 | 1.40 |
| QTIP 2-bit Wino (T6) | 2.60 | 2.00 | 1.90 |
| AQLM 2-bit avg5 (T1) | 5.07 | 4.06 | 1.42 |

to 0.10 at 14B, and at 14B it loses to IQ2_KT by 0.018 nats; the tables above assume nothing about quality, so they say nothing about whether the format keeps a quality edge at larger scale. The 14B encoding time was measured (3.01 h, Table 4); its peak memory and generation speed on a 24 GiB card for the 8B and 14B files were not.

# References


Ben David, N., Meir, O., and Ordentlich, O. OSFP4: Joint optimization of diagonal smoothing and block scales for NVFP4 quantization. *arXiv preprint arXiv:2610.08231*, 2026.

Brown, C. P. Sanskrit prosody and numerical symbols explained. London, 1869.

Chee, J., Cai, Y., Kuleshov, V., and De Sa, C. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2307.13304.

Cheng, W., Zhang, W., Shen, H., Cai, Y., He, X., Lv, K., and Liu, Y. Optimize weight rounding via signed gradient descent for the quantization of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 11332–11350, 2024. arXiv:2309.05516.

Ding, G., Wang, Z., Li, R., and Liu, J. CurveTQ: Rotation-free trellis quantization of LLM weights via curvature-weighted search, 2026. URL `https://arxiv.org/abs/2610.09212`. arXiv:2610.09212.

Dong, Z., Yao, Z., Gholami, A., Mahoney, M., and Keutzer, K. HAWQ: Hessian AWare quantization of neural networks with mixed-precision, 2019. arXiv:1905.03696.

Dutta, A., Krishnan, S., Kwatra, N., and Ramjee, R. Accuracy is not all you need, 2024. arXiv:2407.09141.

Egiazarian, V., Panferov, A., Kuznedelev, D., Frantar, E., Babenko, A., and Alistarh, D. Extreme compression of large language models via additive quantization. In *International Conference on Machine Learning*, 2024. arXiv:2401.06118.

Table 15: Our measured point (MEASURED; ratios CALC). The Llama-3.1-8B build (Appendix F) is not used here.

| Model | ppl: ours / IQ2_XXS | reference ppl | ratio: ours / IQ2_XXS | excess loss remaining |
|---|---|---|---|---|
| Qwen3-4B, B4 | 16.6413 / 28.6083 | 12.4567 (fp16) | 1.336 / 2.297 | 0.259 |
| Qwen3-4B, Model B | 19.0932 / 28.6083 | 12.4567 (fp16) | 1.533 / 2.297 | 0.411 |


Frantar, E., Castro, R. L., Chen, J., Hoefler, T., and Alistarh, D. MARLIN: Mixed-precision auto-regressive parallel inference on large language models, 2024. arXiv:2408.11743.

Guo, H., Brandon, W., Cholakov, R., Ragan-Kelley, J., Xing, E. P., and Kim, Y. Fast matrix multiplications for lookup table-quantized LLMs, 2024. arXiv:2407.10960.

Haase, P., Schwarz, H., Kirchhoffer, H., Wiedemann, S., Marinc, T., Marban, A., Müller, K., Samek, W., Marpe, D., and Wiegand, T. Dependent scalar quantization for neural network compression. In *IEEE International Conference on Image Processing (ICIP)*, pp. 36–40, 2020. doi: 10.1109/ICIP40778.2020.9190955.

Huang, W., Qin, H., Liu, Y., Li, Y., Liu, Q., Liu, X., Benini, L., Magno, M., Zhang, S., and Qi, X. SliM-LLM: Salience-driven mixed-precision quantization for large language models, 2024. arXiv:2405.14917.

Kak, S. Yāmātārājabhānasalagām: An interesting combinatoric sūtra. *Indian Journal of History of Science*, 35:123–127, 2000.

Kawrakow, I. Trellis quantization (IQ2_KT, IQ3_KT, IQ4_KT), ik_llama.cpp pull request 113, 2024. URL `https://github.com/ikawrakow/ik_llama.cpp/pull/113`. Opened 2024-11-15.

Kawrakow, I. New IQ4_KT trellis implementation (integer trellis), ik_llama.cpp pull request 505, 2025a. URL `https://github.com/ikawrakow/ik_llama.cpp/pull/505`. Opened 2025-06-08.

Kawrakow, I. New IQ2_KT, ik_llama.cpp pull request 511, 2025b. URL `https://github.com/ikawrakow/ik_llama.cpp/pull/511`. Opened 2025-06-09, closed in favour of pull request 529.

Kawrakow, I. New IQ2_KT, IQ3_KT and IQ4_KT, V2, ik_llama.cpp pull request 529, 2025c. URL `https://github.com/ikawrakow/ik_llama.cpp/pull/529`. Merged 2025-06-18.

Kawrakow, I. Adding IQ1_KT, 1.75 bpw SOTA quants, ik_llama.cpp pull request 616, 2025d. URL `https://github.com/ikawrakow/ik_llama.cpp/pull/616`. Merged 2025-07-20.

Kawrakow, I. and ik_llama.cpp contributors. ik_llama.cpp: a llama.cpp fork with additional quantization types and improved performance, 2024. URL `https://github.com/ikawrakow/ik_llama.cpp`. Accessed 2026-10-14.

Kim, J., El Halabi, M., Park, W., Schaefer, C. J., Lee, D., Park, Y., Lee, J. W., and Song, H. O. GuidedQuant: Large language model quantization via exploiting end loss guidance. In *International Conference on Machine Learning*, volume 267 of *PMLR*, pp. 30011–30037, 2025. arXiv:2505.07004.

Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., and Keutzer, K. SqueezeLLM: Dense-and-sparse quantization, 2023. arXiv:2306.07629.

Kirchhoffer, H., Haase, P., Samek, W., Müller, K., Rezazadegan-Tavakoli, H., Cricri, F., et al. Overview of the neural network compression and representation (NNR) standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(5):3203–3216, 2022. doi: 10.1109/TCSVT.2021.3095970.

Knuth, D. E. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. Addison-Wesley, 2011.

Kurtic, E., Marques, A., Pandit, S., Kurtz, M., and Alistarh, D. "give me BF16 or give me death"? accuracy-performance trade-offs in LLM quantization, 2024. arXiv:2411.02355.

Lee, D. and Song, H. O. Q-Palette: Fractional-bit quantizers toward optimal bit allocation for efficient LLM deployment, 2025. URL `https://arxiv.org/abs/2509.20214`. arXiv:2509.20214.

Li, X., Chou, T., Fromm, J., Liu, Z., Pan, Y., and Fragouli, C. ScaleBITS: Scalable bitwidth search for hardware-aligned mixed-precision LLMs, 2026. URL https://arxiv.org/abs/2602.17698. arXiv:2602.17698.

Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., and Gu, S. BRECQ: Pushing the limit of post-training quantization by block reconstruction, 2021. arXiv:2102.05426.

Lifar, E., Savkin, S., Ordentlich, O., and Polyanskiy, Y. WaterSIC: Information-theoretically (near) optimal linear layer quantization. *arXiv preprint arXiv:2603.04956*, 2026.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024a. arXiv:2306.00978.

Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., and Han, S. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving, 2024b. arXiv:2405.04532.

Liu, Y., Wen, J., Wang, Y., Ye, S., Zhang, L. L., Cao, T., Li, C., and Yang, M. VPTQ: Extreme low-bit vector post-training quantization for large language models, 2024. arXiv:2409.17066.

Malandrino, P.-J. Tetra: Serving Leech-lattice quantized LLMs at 2.7 bits per parameter, 2026. URL https://arxiv.org/abs/2609.35465. arXiv:2609.35465.

Malinovskii, V., Panferov, A., Ilin, I., Guo, H., Richtárik, P., and Alistarh, D. HIGGS: Pushing the limits of large language model quantization via the linearity theorem. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10857–10886, 2025. arXiv:2411.17525.

Marcellin, M. W. and Fischer, T. R. Trellis coded quantization of memoryless and Gauss-Markov sources. *IEEE Transactions on Communications*, 38(1):82–93, 1990.

Narita, Y. and Sato, I. Understanding LLM quantization through activation-guided compensation and orthogonal residuals. *arXiv preprint arXiv:2609.21450*, 2026.

Ordentlich, O. and Polyanskiy, Y. High-rate quantized matrix multiplication II. *arXiv preprint arXiv:2605.13768*, 2026.

Panter, P. F. and Dite, W. Quantization distortion in pulse-count modulation with nonuniform spacing of levels. *Proceedings of the IRE*, 39(1):44–48, 1951.

Park, G., Park, B., Kim, M., Lee, S., Kim, J., Kwon, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models, 2022. arXiv:2206.09557.

Pingala. *Chandahśāstram of Śrī Piṅgalanāga, with the commentary Mrtasañjīvanī of Halāyudha Bhaṭṭa*. Number 91 in Kāvyamālā. Nirnaya Sagar Press, Bombay, 3rd edition, 1938. Edited by Paṇḍit Kedāranāth of Jaipur; 3rd edition, revised with notes by Anant Yajñeśvar Śāstrī Dhupkar.

Plofker, K. *Mathematics in India*. Princeton University Press, Princeton, NJ, 2009.

Que, X., Elkayam, N., Pyakurel, S., Pan, S., and Gope, D. XOR-Trellis: Ultra-low-complexity dequantization and curvature-aware Hadamard-free LLM quantization, 2026.

Sieberling, O., Kuznedelev, D., Kurtic, E., and Alistarh, D. EvoPress: Accurate dynamic model compression via evolutionary search, 2024. arXiv:2410.14649.

Tseng, A., Sun, Q., Hou, D., and De Sa, C. QTIP: Quantization with trellises and incoherence processing. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2406.11235.

Tseng, A., Sun, Z., and De Sa, C. Model-preserving adaptive rounding. In *International Conference on Machine Learning*, 2026. arXiv:2505.22988.

turboderp-org. ExLlamaV3: EXL3 quantization, a streamlined variant of QTIP, 2025. URL https://github.com/turboderp-org/exllamav3. Repository README, section "EXL3 quantization". Accessed 2026-10-14.

Ungerboeck, G. Channel coding with multilevel/phase signals. *IEEE Transactions on Information Theory*, 28(1):55–67, 1982.

van Baalen, M., Kuzmin, A., Koryakovskiy, I., Nagel, M., Couperus, P., Bastoul, C., Mahurin, E., Blankevoort, T., and Whatmough, P. GPTVQ: The blessing of dimensionality for LLM quantization, 2024. arXiv:2402.15319.

van Nooten, B. Binary numbers in Indian antiquity. *Journal of Indian Philosophy*, 21:31–50, 1993.

Wei, J., Cao, S., Cao, T., Ma, L., Wang, L., Zhang, Y., and Yang, M. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge, 2024. arXiv:2407.00088.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, 2023. arXiv:2211.10438.

Yan, X., Wu, Z., Zhang, K., Yang, K., and Zhang, Y. SAGE: Pushing 2-bit vector-quantized LLMs toward real-world usability, 2026. URL https://arxiv.org/abs/2610.14745. arXiv:2610.14745.

Yang, Z., Li, X., Cheng, K., Liu, K., et al. Proteus: Lookup-free trellis-coded quantization by lattice-breaking compute codes for 2-bit LLMs. International Conference on Machine Learning (ICML) 2026, poster, 2026. URL https://icml.cc/virtual/2026/poster/63948.

Yuan, Z., Shang, Y., Zhou, Y., Dong, Z., Zhou, Z., Xue, C., Wu, B., Li, Z., Gu, Q., Lee, Y. J., Yan, Y., Chen, B., Sun, G., and Keutzer, K. LLM inference unveiled: Survey and roofline model insights, 2024. arXiv:2402.16363.

Zhang, H., Zhang, Q., Cao, Q., Fan, H., Huang, Y., Sha, K., and Feng, Y. AlignQuant: Tile-aligned mixed-precision quantization for efficient LLM generation, 2026. URL https://arxiv.org/abs/2610.07457. arXiv:2610.07457.

Zheng, X., Li, Y., Chu, H., Feng, Y., Ma, X., Luo, J., Guo, J., Qin, H., Magno, M., and Liu, X. An empirical study of Qwen3 quantization, 2025. arXiv:2505.02214.

Zhou, Z., Li, X., Li, M., Zhang, H., Wang, H., Chang, W., Liu, Y., Dang, Q., Yu, D., Ma, Y., and Wang, H. CCQ: Convolutional code for extreme low-bit quantization in LLMs, 2025. arXiv:2507.07145.